



FC7xxx MCAL User Manual

Rev: 1.5.1

Target Devices:

FC7300F8MDQxXxxxT1B, FC7300F8MDTxXxxxT1B
FC7300F4MDDxXxxxT1B, FC7300F4MDDxXxxxT1C
FC7300F4MDSxXxxxT1B, FC7300F4MDSxXxxxT1C
FC7240F2MDSxXxxxT1A

About This Document

Introduction

This User Manual is intended to enable users to integrate the Microcontroller Abstraction Layer (MCAL) software for the FC7XXX microcontroller.

This document provides detailed information on package installation, project using(based on Mcal_Demo_Board_Project_FC7XXX attached) and other generic information that are common across all the modules.



Notification

Detailed information about each module are provided in module specific user manuals.

Intended Audience

This document is intended for anyone using the FC7XXX MCAL software.

Table of Contents

About This Document	I
Introduction	I
Intended Audience	I
Chapter 1. Introduction	1
1.1. About This Manual	1
1.2. Document Conventions	1
1.3. Acronyms and Abbreviations	1
1.4. Purpose	3
1.5. Scope	3
Chapter 2. Safety MCAL Package	5
2.1. MCAL Component	5
2.2. SW package Content	7
Chapter 3. MCAL Usage	10
3.1. Configuration Tools Installation	10
3.1.1. EB tresos Installation	10
3.1.2. EB Client License Administrator Installation	11
3.1.3. EB License Activation	12
3.1.3.1. Online Activation	12
3.1.3.2. Offline Activation	13
3.1.4. MCAL Plugins Installation	14
3.1.5. MCAL Plugins Update	16
3.2. MCAL Project Creation	16
3.2.1. Create an EB tresos Project from Scratch	16
3.2.2. Import an Existing EB tresos Project	19
3.3. MCAL Project Integration	21
3.3.1. Integration of Configurations with Static Code	21
3.3.2. Build and Debug	22
3.3.2.1. Build and Debug with Flagchip FC_IDE	22
3.3.2.2. Build and Debug with IAR Embedded Workbench	25
Chapter 4. Extra Feature beyond AUTOSAR R20-11	28
Chapter 5. ECU System Environment	30
5.1. MCAL Version and Variant	30
5.2. System Safe State	30
5.3. ECU System Software Architecture	30
5.4. Multicore Considerations	32
5.4.1. Multicore Startup Procedure	33
5.4.2. Multicore De-Initialization	33
5.4.3. Multicore Interrupts	34
5.4.4. Data Consistency in Multicore System	34
5.4.4.1. Multicore Shared Data Consistency	34
5.4.4.2. Data Consistency when Cache Enabled	35
Chapter 6. AUTOSAR Ecosystem	38
6.1. AUTOSAR BASIC SW Package	38
6.1.1. MICROSAR Classic Vector SLP4	38

6.1.2. INTEWORK-EAS-CP	39
6.1.3. ORIENTAIS Classic AUTOSAR (CECT)	39
6.1.4. NeuSAR cCore (Neusoft REACH)	40
6.2. MCU Safety Library	41
6.2.1. FC7XXX Safety Frame (ZhiCong)	41
Chapter 7. MCAL safety mechanisms	42
Chapter 8. MCU safety mechanisms	43
Chapter 9. Known Issues	44
Chapter 10. Deviations and Limitations	45
Chapter 11. Revision History	46

Chapter 1. Introduction

1.1. About This Manual

This User Manual describes the Flagchip AutoSAR Microcontroller Abstraction Layer (MCAL) software driver for the FC7XXX series of microcontrollers.

1.2. Document Conventions

This manual uses the following conventions:

- **Bold** is used to indicate the important terms, notes and warnings.
- *Italic* is used to indicate the file names.
- Monospace is used to indicate the code, commands, and other text that should be typed literally by the user.

Notes and warnings are shown as below:



Notification

This is a note.



Warning

This is a warning.

1.3. Acronyms and Abbreviations

Table 1: Acronyms and Abbreviations

Acronym	Abbreviation
MCAL	Microcontroller Abstraction Layer
SWS	Software Specification
API	Application Programming Interface
BSW	Basic Software
EB	Elektrobit (sometimes means EB tresos)
IDE	Integrated Development Environment
ADC	Analog to Digital Converter
CAN	Controller Area Network
Common	The Resource and Common files for all modules
CRYPTO	Crypto module
CRYIF	Crypto Interface
DEM	Diagnostic Event Manager
DET	Default Error Tracer
DIO	Digital Input Output
DMA	Direct Memory Access

Acronym	Abbreviation
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read-Only Memory
ETH	Ethernet Controller Driver
ETHIF	Ethernet Interface
ETHTRCV	Ethernet Transceiver Driver
FEE	Flash EEPROM Emulation
FLS	Flash Loader System
PFLS	Programming Flash Loader System
GPT	General Purpose Timer
I2C	Inter-Integrated Circuit
I2S	Inter-IC Sound
ICU	Input Capture Unit
IoHwAb	I/O Hardware Abstraction
IRQ	Interrupt Request
ISR	Interrupt Service Routine
LIN	Local Interconnect Network
LINIF	LIN Interface
MB	Mailbox (interface for multiple cores to communicate and synchronize)
MCU	Microcontroller Unit
MPU	Memory Protection Unit
MSC	Micro Second Channel
NVM	Non-Volatile Memory Manager
OCU	Output Compare Unit
OS	Operating System
PB Variant	Post Build Variant
PC Variant	Pre-Compile Variant
PDU	Protocol Data Unit
PORT	Module for configuration of port and Pinmux
PWM	Pulse Width Modulation
SDADC	Sigma Delta Analog to Digital Converter
SENT	Single Edge Nibble Transmission
SPI	Serial Peripheral Interface
SSI	Speed and Sensor Interface
TRGSEL	Trigger Select
TRNG	True Random Number Generator
UART	Universal Asynchronous Receiver/Transmitter
WDG	Watchdog
WDGIF	Watchdog Interface

1.4. Purpose

This document gives an overview on product-wide behaviour or use-cases.

The MCAL development typically starts earlier in time compared to integrator system development and MCAL will finally be used in multiply systems. Hence it is a generic software element, during the development of a general market ECU software, at the start of development stage, this document defines the safety objectives, architectural assumptions and the AoUs to be fulfilled by the integrator for the usage of MCAL drivers in safety relevant applications.

To ensure correct operation of the product always follow the instructions provided in technical references and (additionally for safety projects) safety manual.

1.5. Scope

The MCAL modules apply to Flagchip FC7300/FC7240 series MCUs driver development. The series include ICs as follows:

- FC7300F8MDQ, FC7300F8MDT, FC7300F4MDD, FC7300F4MDS:
 Each FC7300 variant contains 320 pin,176pin package.
 Each FC7300 has core number as quad-core(Q), tri-core(T), dual-core(D), single-core(S). with internal flash size difference as 8MB and 4MB.
- FC7240F2MDS:
 FC7240 variant has single core(S) and contains 176pin,144pin and 100pin package.

Also derived to series that support different features like different instance of CAN, ADC...

To support these variants user, shall configure the IC device type in common module. This document is applied for IC variants as following table:

Project Name	FC7XXX MCAL
Project Brief	FC7XXX MCAL modules with ISO26262 function safety ASIL D
Target IC Variants	This FC7XXX MCAL SW package apply to FC7XXX MCU series. Including the IC variants: • FC7240F2MDS1P100T1A • FC7240F2MDS1P144T1A • FC7240F2MDS1P176T1A • FC7300F4MDD1A320T1B • FC7300F4MDD1P176T1B • FC7300F4MDD2A320T1B • FC7300F4MDD2P176T1B • FC7300F4MDD3A320T1B • FC7300F4MDS1A320T1B • FC7300F4MDS1P176T1B • FC7300F4MDS2A320T1B • FC7300F4MDS2P176T1B • FC7300F8MDT2A320T1B • FC7300F8MDT2P176T1B • FC7300F8MDQ1A320T1A

- | | |
|--|---|
| | <ul style="list-style-type: none">• FC7300F8MDQ1A257T1A• FC7300F8MDQ1P176T1A• FC7300F4MDD1A180T1C• FC7300F4MDD1A320T1C• FC7300F4MDS1A180T1C• FC7300F4MDS1A320T1C |
|--|---|

Chapter 2. Safety MCAL Package

2.1. MCAL Component

FC7XXX MCAL modules deliver ISO 26262 functional safety ASIL D level drivers to customer. For integrator to developer ECU SW that apply to AUTOSAR architecture. Figure 1 shows the modules provided in our MCAL package according to the AutoSAR layered architecture.

Modules colored with ■ are tested modules and could be used for production purpose while modules colored with □ are stub modules and shall not be used in production.

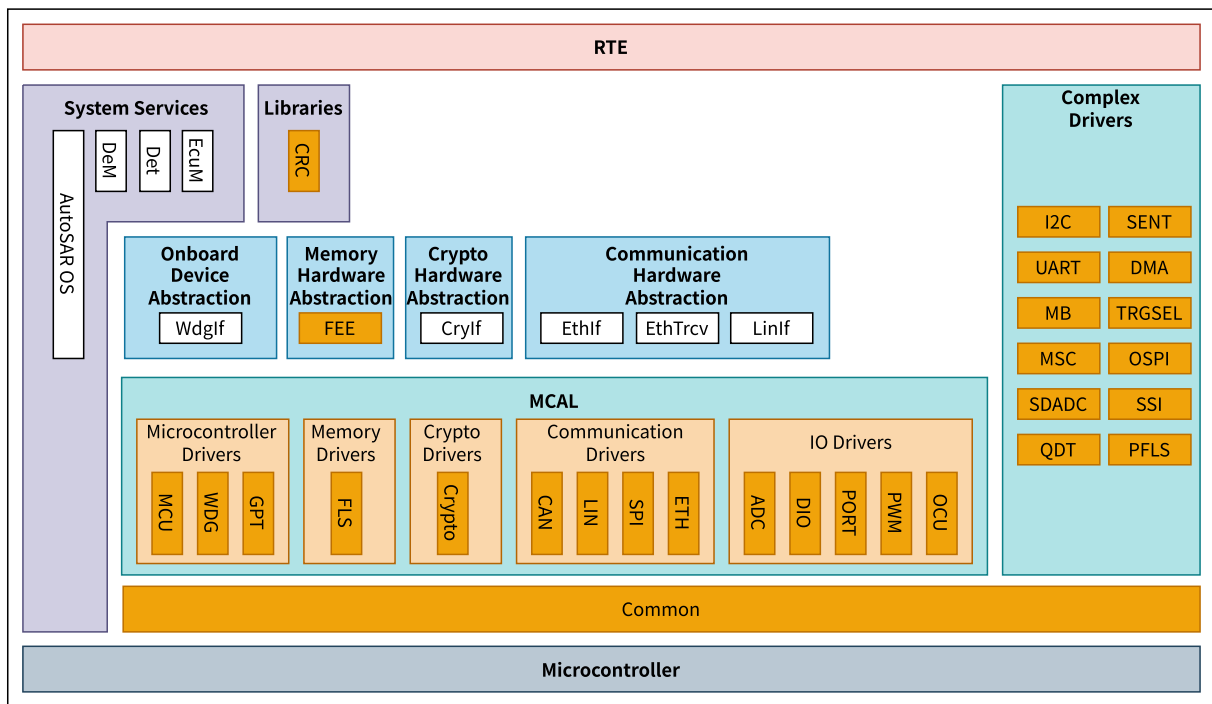


Figure 1: MCAL Scope

the FC7XXX MCAL software package, contains the following modules:

- Microcontroller drivers:
 - Microcontroller Unit (MCU) driver
 - General Purpose Timer (GPT) driver
 - Watchdog (WDG) driver for the internal CPU watchdogs
- Memory & Memory hardware abstraction drivers
 - Internal Flash (FLS) driver
 - Flash EEPROM Emulation (FEE) driver
- Crypto drivers
 - Crypto driver
- Communication drivers
 - Controller Area Network (CAN) driver
 - Local Interconnect Network (LIN) driver
 - Serial Peripheral Interface (SPI) driver
 - Ethernet (ENET) driver
- I/O drivers

- Port (PORT) driver
- Digital I/O (DIO) driver
- Analog-to-Digital Converter (ADC) driver
- Pulse Width Modulation (PWM) driver
- Input Capture Unit (ICU) driver
- Output Compare Unit (OCU) driver
- Library
 - Cyclic Redundancy Check (CRC) driver
- Complex drivers
 - Direct Memory Access (DMA) driver
 - Inter-Integrated Circuit (I2C) driver
 - Trigger Select (TRGSEL) driver
 - Interface for multiple cores to communicate and synchronize (MAILBOX) driver
 - Micro Second Channel (MSC) driver
 - Single Edge Nibble Transmission (SENT) driver
 - Universal Asynchronous Receiver Transmitter (UART) driver
 - Octal Serial Peripheral Interface (OSPI) driver
 - Sigma-Delta Analog-to-Digital Converter (SDADC) driver
 - Speed and Sensor Interface (SSI) driver
 - Quadrature Decoder Timer (QDT) driver
 - Programmable Flash Loader (PFLS) driver

Notification

Not all modules are available in every chip derivative.

The modules overview is shown in Table 2.

Table 2: List of Production Modules

No	Modules	Multi-Core Type	Functional Group	ASIL Level
1	MCU	Type I	Microcontroller Drivers	ASIL D
2	WDG	Type II		ASIL D
3	GPT	Type II		ASIL D
4	FLS	Type I	Memory Drivers	ASIL D
5	Crypto	Type III	Crypto Drivers	ASIL D
6	CAN	Type II	Communication Drivers	ASIL D
7	LIN	Type II		ASIL D
8	SPI	Type II		ASIL D
9	ENET	Type II		ASIL D
10	ADC	Type II	I/O Drivers	ASIL D
11	DIO	Type III		ASIL D
12	PORT	Type I		ASIL D
13	PWM	Type II/Type III		ASIL D

No	Modules	Multi-Core Type	Functional Group	ASIL Level
14	ICU	Type II	Complex Drivers	ASIL D
15	OCU	Type II		ASIL D
16	DMA	Type II/Type III		ASIL D
17	I2C	Type II		ASIL D
18	TrgSel	Type I		ASIL D
19	MB	Type III		ASIL D
20	MSC	Type II		ASIL D
21	SENT	Type II		ASIL D
22	UART	Type II		ASIL D
23	OSPI	Type I		ASIL D
24	SDADC	Type II		ASIL D
25	SSI	Type II		ASIL D
26	QDT	Type II		ASIL D
27	PFLS	Type I		ASIL D
28	CRC	Type I	Library	ASIL D
29	FEE	Type I	Memory Hardware Abstraction	ASIL D

Integrator shall configure MCAL modules and together integrate other AUTOSAR BSW modules in application project. Flagchip provides stub modules for MCAL configuration and code project bring up as a standalone software package. These stub modules do not provide original full functionality, only provide the ECUC settings that would be referred by MCAL modules. Therefore, these parts are not considered in MCAL package scope.

integrator shall replace these modules with AUTOSAR BSW modules.

Detailed stub modules list as following:

No	Modules	Module Type	Functional Group
30	WdgIf	Stub Module	Onboard Device Abstraction
31	CryIf	Stub Module	Crypto Hardware Abstraction
32	EthIf	Stub Module	Communication Hardware Abstraction
33	EthTrcv	Stub Module	
34	LinIf	Stub Module	
35	Dem	Stub Module	System Service Stub
36	Os	Stub Module	

2.2. SW package Content

The design of FC7XXX MCAL follows the specifications of AUTOSAR Classic Platform Release 4.6.0. All files provided in the package have structure shown in Figure 2.

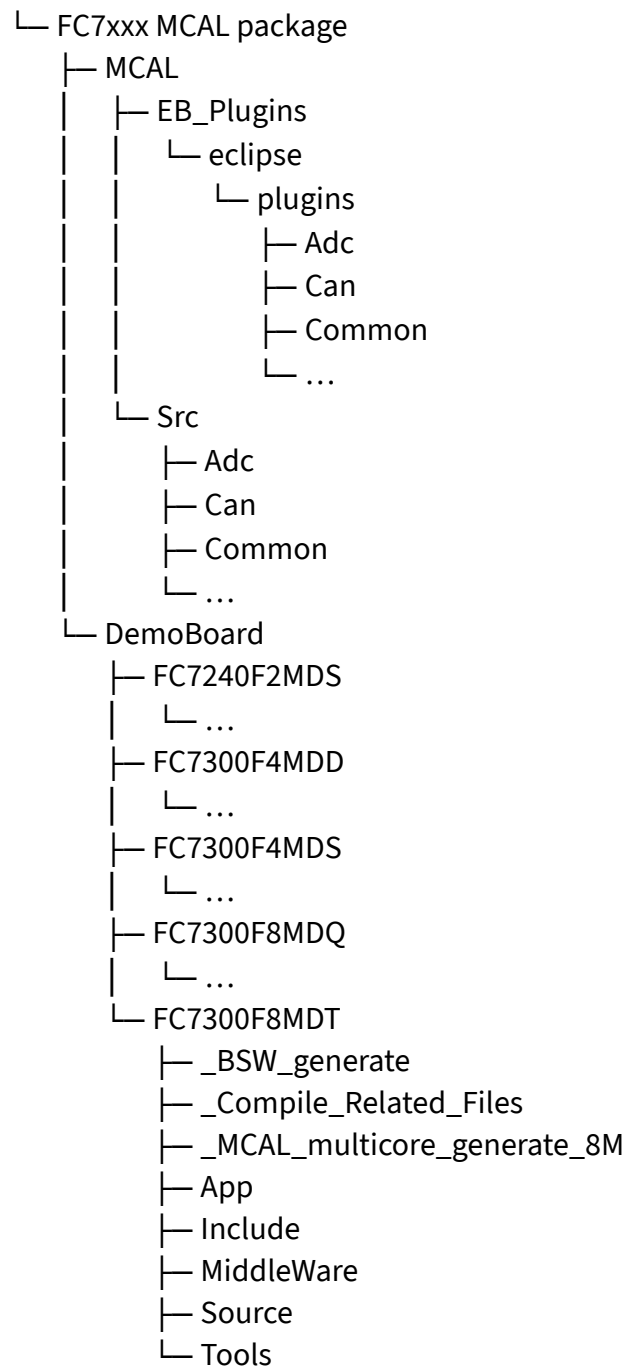


Figure 2: File Structure

The files are grouped in two folders. The MCAL folder for MCAL plugins and static code. The DemoBoard folder for MCAL demos.

MCAL/EB_Plugins: EB Tresos plugins of all MCAL modules include UM & IM docs, module configuration files and template files

MCAL/Src: Static source code of all MCAL modules

Demoboard: Examples of all chip variants

Each MCAL plugin in the MCAL/EB_Plugins folder contains the following contents:

autosar: The AutoSAR arxml model of the module

config: The module GUI configuration template for EB tresos and some pre-defined configurations

doc: User manual and integration manual of the module

META-INF: Meta info and signature of the module

template: Code generation template

Notification

Please **do not** put the MCAL/EB_Plugins in the folder which is managed by SCM tools such as Git or SVN. Because some source code management tools will change the line endings of files, and this will result in the corruption of the signature.

In the DemoBoard folder, there are sub folders of examples for all chip variants. Take the FC7300F8MDT as an example, below is the description of all folders inside the directory.

- _BSW_generate: Some stub files that should be generated by BSW modules
- _Compile_Related_Files: Some common files such as systick and Det driver
- _MCAL_multicore_generate_8M: Generated files from EB tresos of the MCAL modules
 - App: The simple usage demo of MCAL modules
 - Include: The header file of some common modules
 - Source: The main entry and the source file of some common modules
 - MiddleWare: Some middle ware for semi-hosting and MPU configuration
 - tools: Project files for different tools
 - tools/EB_Project: Project files and configurations for EB tresos
 - tools/FC_Project: Project files and startup and link scripts for Flagchip FC_IDE
 - tools/GHS_FC_Project: Project files and startup and link scripts for GreenHills eclipse plugin (integrated in Flagchip FC_IDE)
 - tools/GHS_Project: Project files and startup and link scripts for GreenHills Multi IDE
 - tools/HighTec_Project: Project files and startup and link scripts for HighTec Development Platform
 - tools/IAR_Project: Project files and startup and link scripts for IAR Embedded Workbench

Notification

The executable file(elf/hex/bin) will be generated after compiling each project in the DemoBoard/tools/xxProject/.. path.

Notification

The files in DemoBoard folder are non-productive source files and these files contains references to test code. The integrator shall take care if any of the code in respective subfolder files are not applicable for the end application.

Chapter 3. MCAL Usage

Normally, the usage of FC7XXX MCAL involves following steps:

- Install Elektrobit Automotive Software for AUTOSAR automotive development: EB tresos studio.
- Import plugins of FC7XXX MCAL and generate configuration codes for each module based on EB tresos Studio.
- Integrate MCAL source code, generated configuration code and other AUTOSAR layers codes to user's projects.
- Compile and download to the FC7XXX chip.

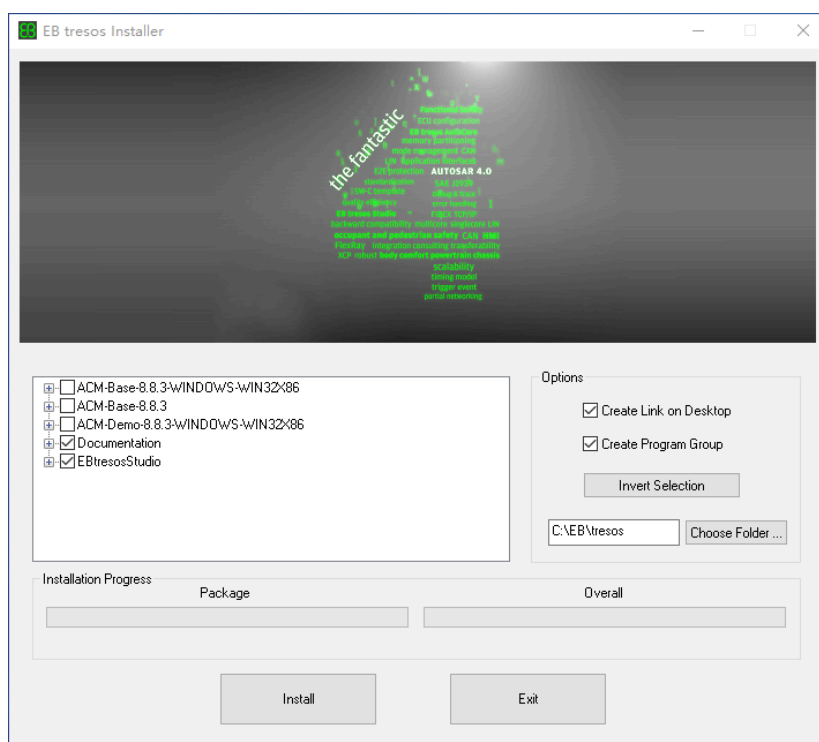
3.1. Configuration Tools Installation

Components of the FC7XXX MCAL have been developed as plugins of EB tresos 28.1.0, and is known to work on EB tresos 28.1.0 to EB tresos 29.2.0. If it is not yet installed on your computer, you could download it from EB official website (<https://www.elektrobit.com>) or our release site (<https://tools.flagchip.com.cn/>).

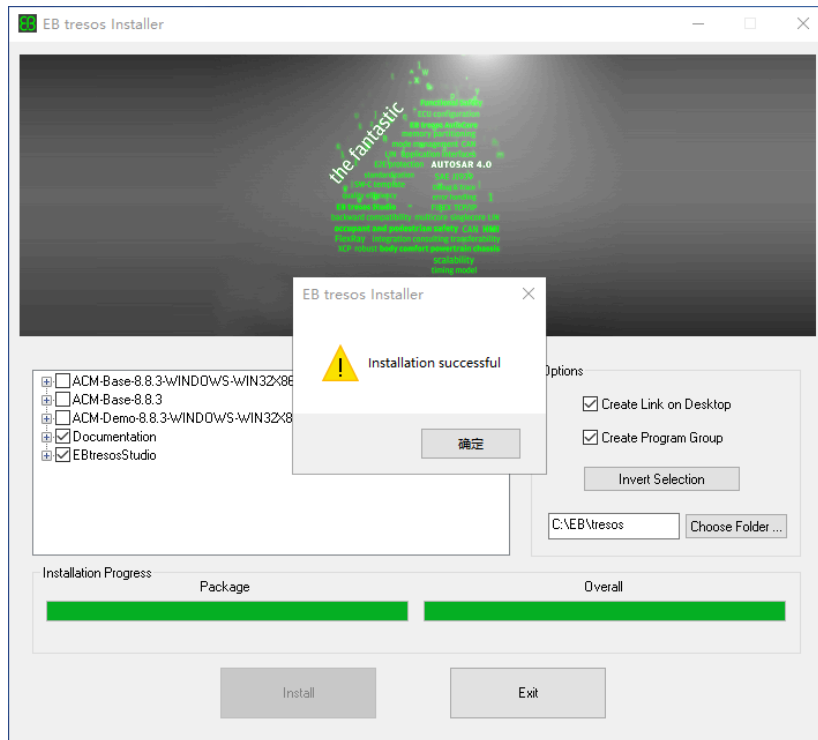
Besides EB tresos, you shall also install Elektrobit Client License Administrator to active the software. You could receive our activation code from our sales team or our field support team.

3.1.1. EB tresos Installation

1. Extract the downloaded file and click **steup.exe** in the extracted folder.
2. Select the components and folder to install.



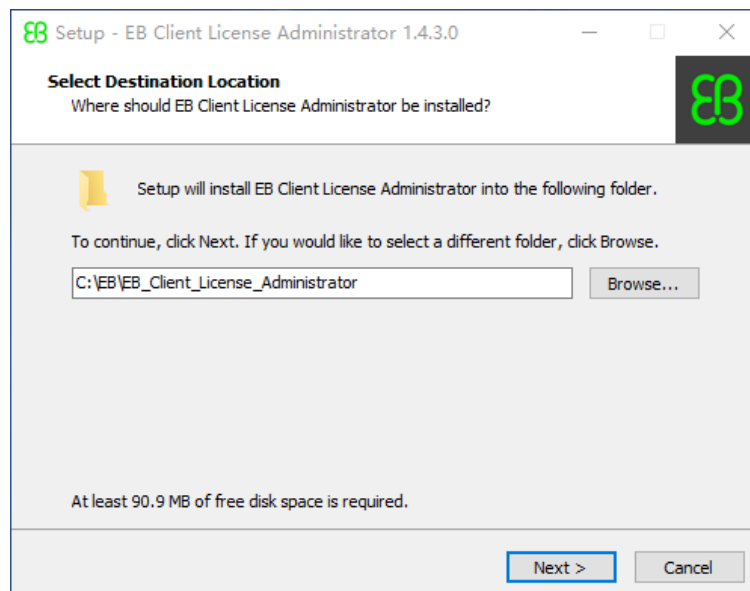
3. Wait the installation process to finish.



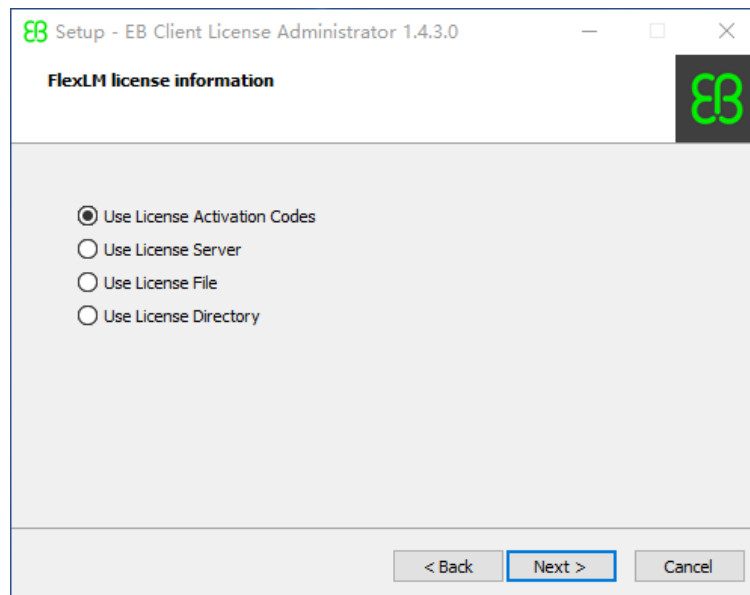
3.1.2. EB Client License Administrator Installation

You must use an EB Client License Administrator v1.4.3 and above for our MCAL modules.

1. Extract the downloaded file and click **EB_Client_License_Administrator_1_4_3_Setup.exe** in the extracted folder.
2. Select the installation folder and other configurations.



3. In the **FlexLM license information** tab, select **Use License Activation Codes** option.



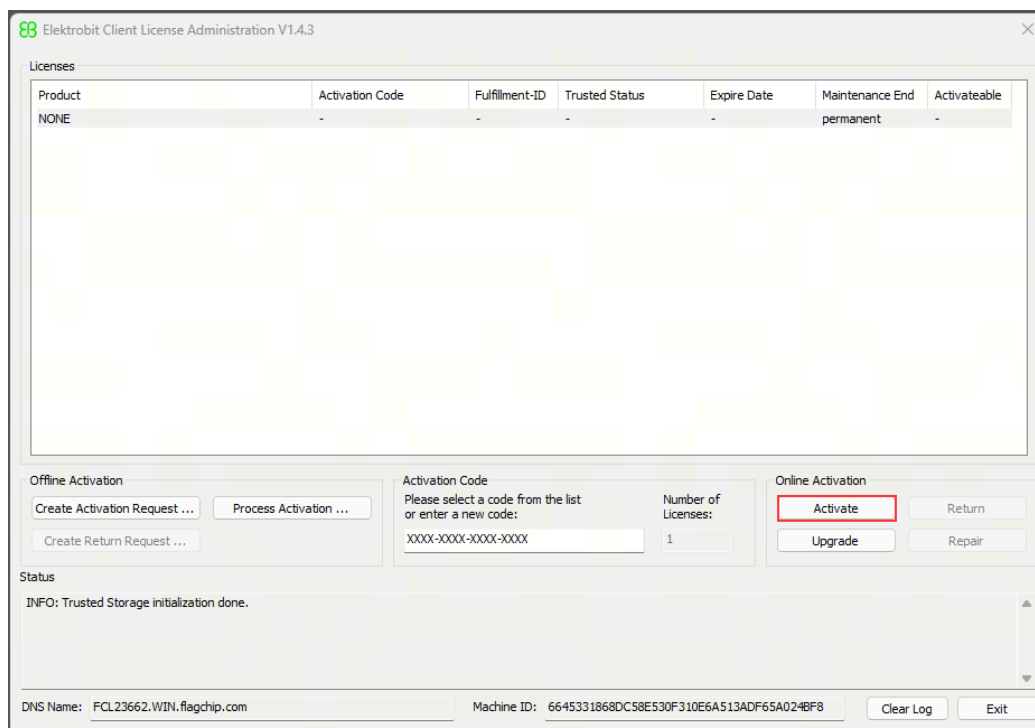
4. Click **Install** and wait the installation to be finished.

3.1.3. EB License Activation

EB tresos shall be activated before using. You must use the activation code provided by Flagchip to use the MCAL modules.

3.1.3.1. Online Activation

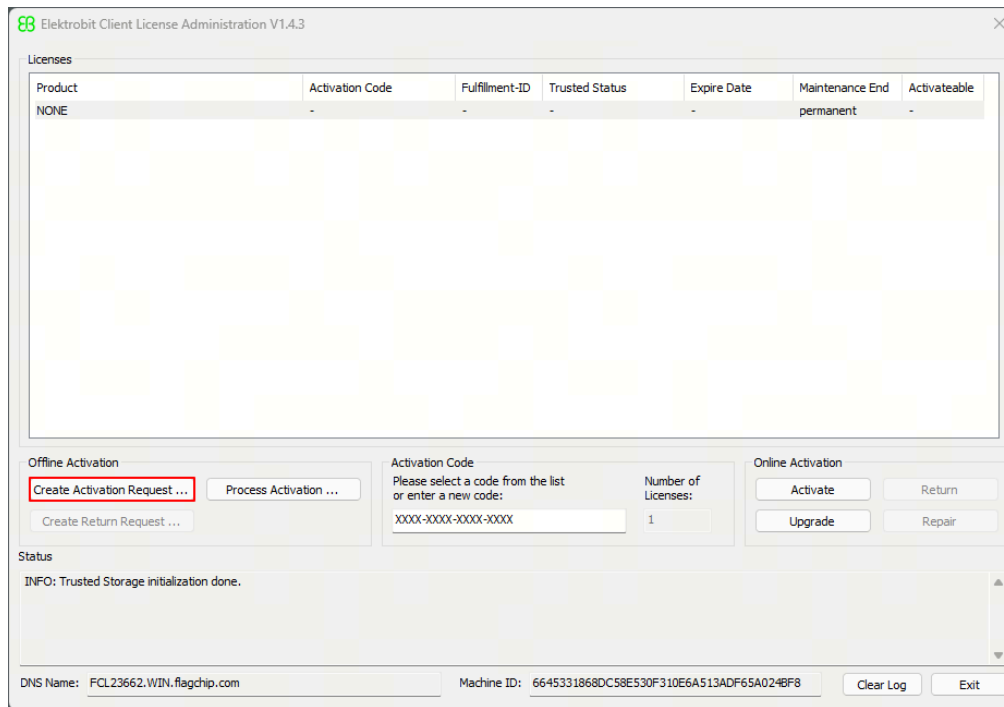
If internet is accessible, the easiest way to active EB tresos is to use the online activation. Just input the activation code (it should be in the form of XXXX-XXXX-XXXX-XXXX) in the **Activation Code** box and then press the **Activate** button, the software will be activated automatically.



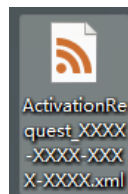
3.1.3.2. Offline Activation

If internet is not available, you could use the offline activation. The steps for offline activation is as follows.

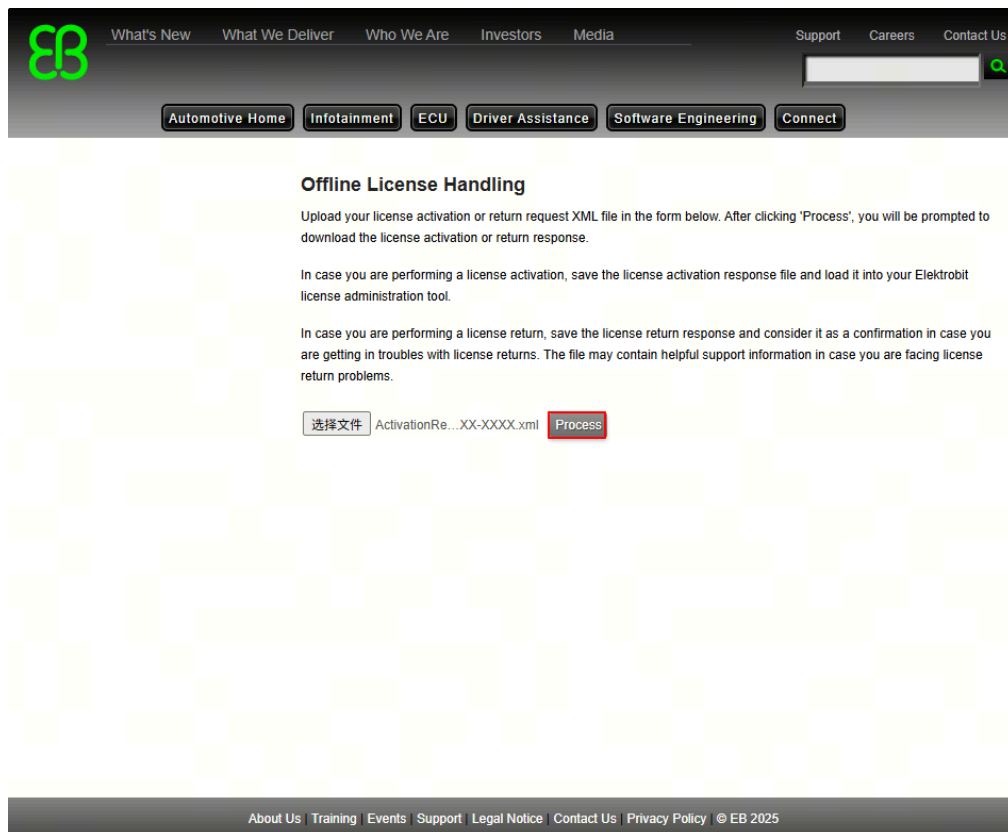
1. Input the activation code in the **Activation Code** box and then press **Create Activation Request...** button



2. Then you will get an activation request file.



3. Send the file to a computer which could access internet and open the EB offline activation website (<https://elektrotbit.flexnetoperations.com/control/elkb/offlineActivation>). Upload the file and then click **Process**.



Offline License Handling

Upload your license activation or return request XML file in the form below. After clicking 'Process', you will be prompted to download the license activation or return response.

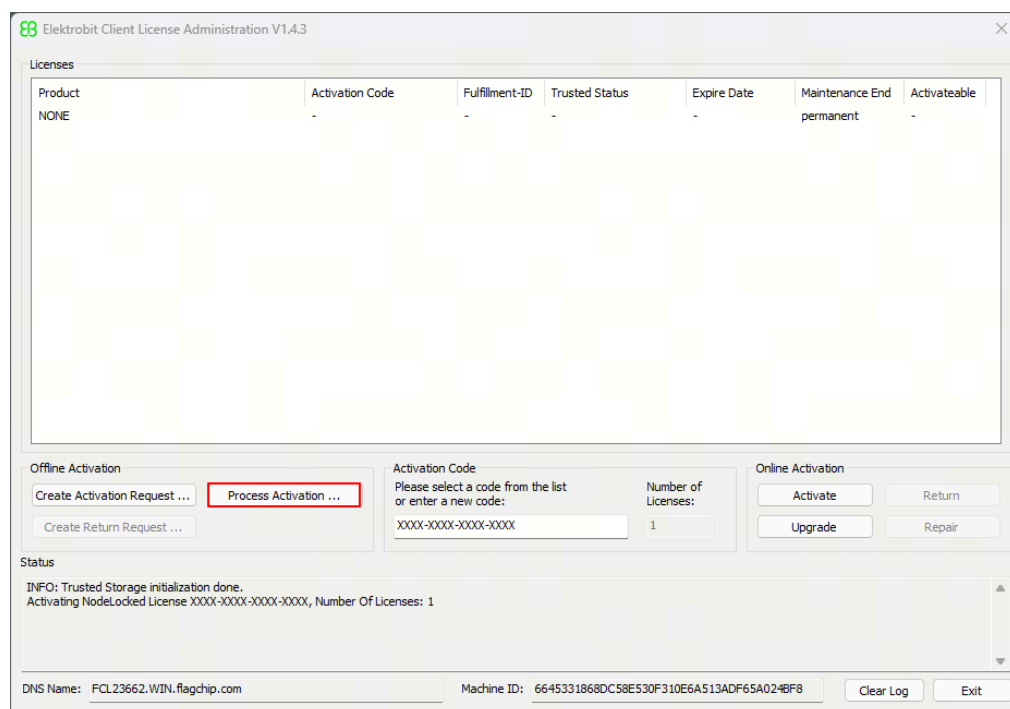
In case you are performing a license activation, save the license activation response file and load it into your Elektrotit license administration tool.

In case you are performing a license return, save the license return response and consider it as a confirmation in case you are getting in troubles with license returns. The file may contain helpful support information in case you are facing license return problems.

选择文件 ActivationRe...XX-XXXX.xml **Process**

About Us | Training | Events | Support | Legal Notice | Contact Us | Privacy Policy | © EB 2025

- After that, you will get an activation response file. Send the response file to the machine which installed EB tresos.
- Click **Process Activation...** and select the response file. After that the software will be activated.



Elektrotit Client License Administration V1.4.3

Product	Activation Code	Fulfillment-ID	Trusted Status	Expire Date	Maintenance End	Activateable
NONE	-	-	-	-	permanent	-

Offline Activation

Create Activation Request ... **Process Activation ...** Create Return Request ...

Activation Code
Please select a code from the list or enter a new code:
XXXX-XXXX-XXXX-XXXX

Number of Licenses:
1

Online Activation

Activate Return Upgrade Repair

Status
INFO: Trusted Storage initialization done.
Activating NodeLocked License XXXX-XXXX-XXXX-XXXX, Number Of Licenses: 1

DNS Name: FCL23662.WIN.flagchip.com Machine ID: 6645331868DC58E530F310E6A513ADF65A024BF8 Clear Log Exit

3.1.4. MCAL Plugins Installation

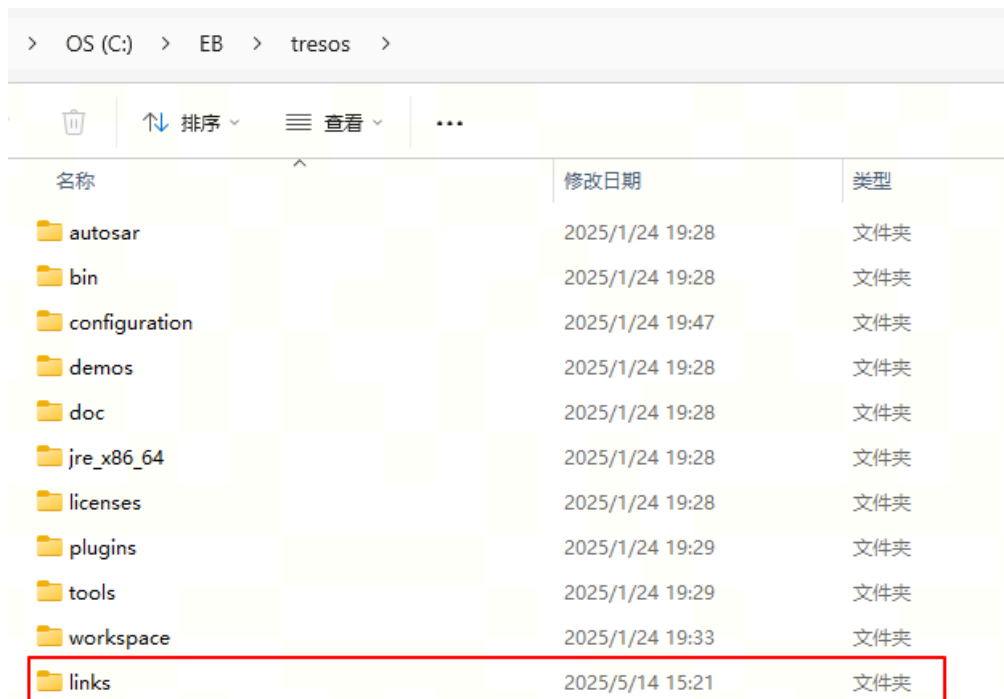
After extract the MCAL package, you will get a folder with two folders: **DemoBoard** and **MCAL**. The MCAL plugins are contained in the MCAL\EB_Plugins folder.

Let's assume that you have extracted the MCAL package into the C:\Flagchip\FC7XXX_MCAL\ folder, as shown in Figure 3. Then the plugins will be in the C:\Flagchip\FC7XXX_MCAL\MCAL\EB_Plugins folder. The easiest way to install the MCAL plugins is to create a link file pointing to the plugins folder in EB tresos.

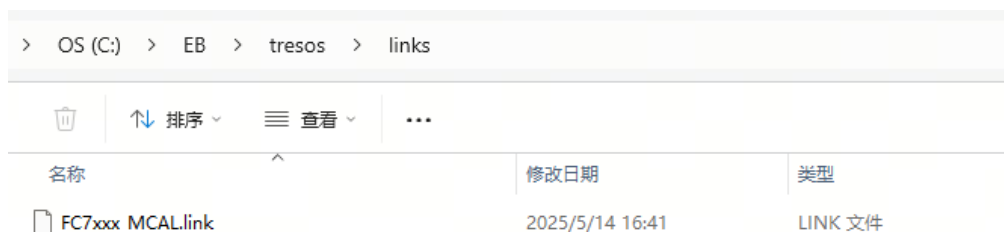


Figure 3: FC7xxx MCAL Package

1. Create a **links** folder in the root folder of EB tresos.



2. Make a **.link** file in the links folder, for example, FC7xxx_MCAL.link.



3. Put path= with the EB plugins folder path in the link file. Since Windows system uses backslash(\) for folder, you shall substitute the backslash(\\) with slash(/) in the path. For example, if the EB plugins folder of our MCAL is in the C:\Flagchip\FC7XXX_MCAL\MCAL\EB_Plugins folder, then the contents in the FC7xxx_MCAL.link file should be:

path=C:/Flagchip/FC7XXX_MCAL/MCAL/EB_Plugins

After these steps, the MCAL plugins are successfully registered in the EB tresos and you can make an EB tresos project with our MCUs.

Besides the method of creating the link file, you could also copy the plugins into the plugins folder in EB tresos.

3.1.5. MCAL Plugins Update

In order to update the MCAL plugins, you could substitute the old plugins with new released plugins, or you could change the link file to point to the new released MCAL plugins folder.

After update the MCAL plugins, you should clear the EB tresos cache files for the plugins to work properly. The cache files are in the **configuration** folder in the EB tresos installation folder. You shall remove the folders shown in the red box in Figure 4 to clear the cache.

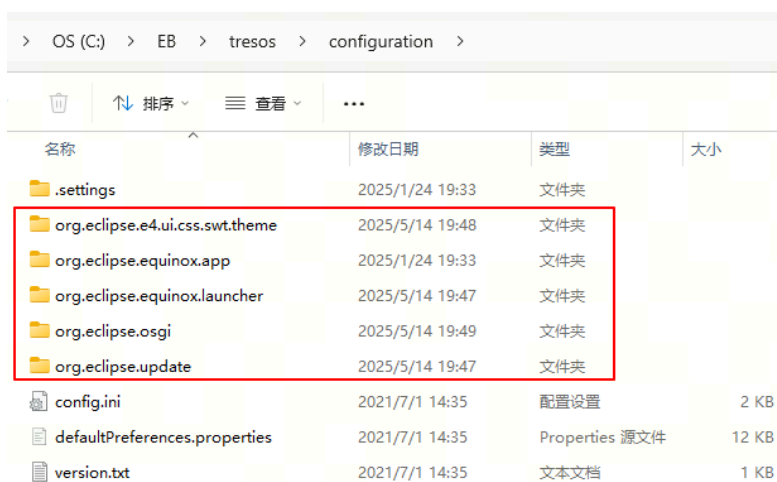
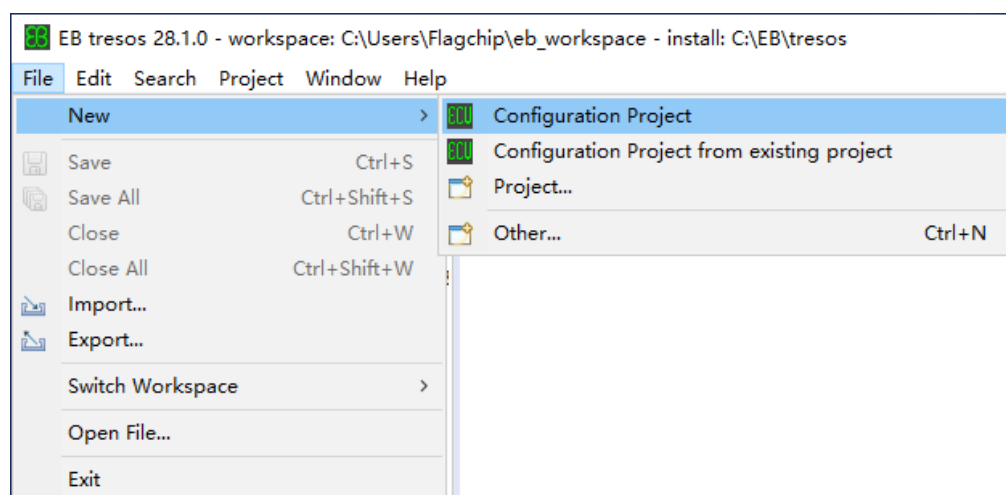


Figure 4: EB Cache Files

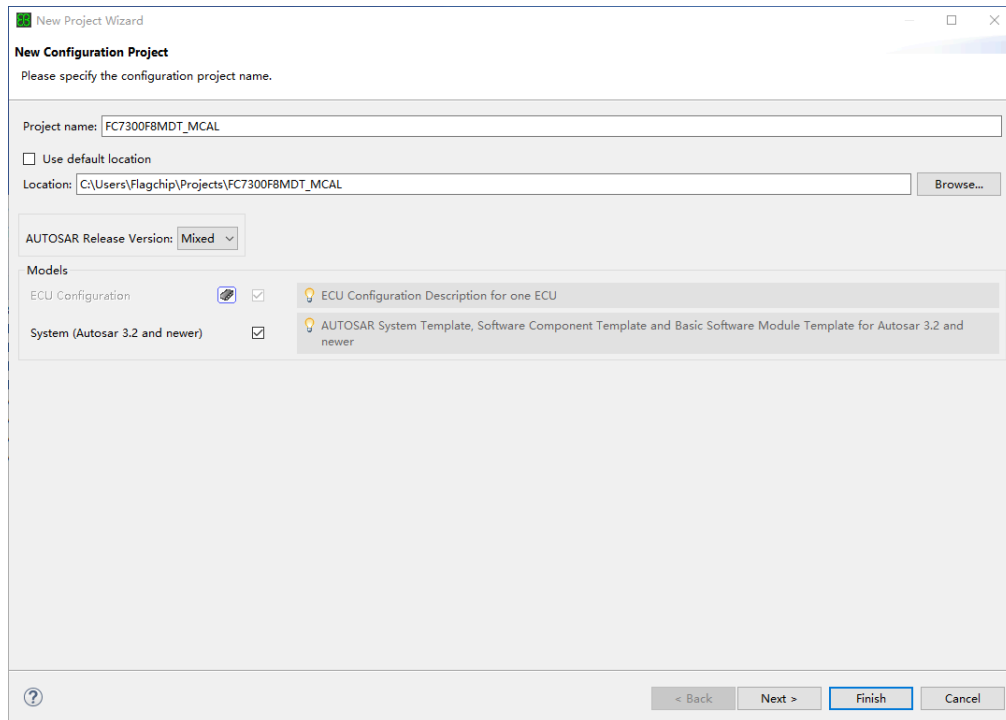
3.2. MCAL Project Creation

3.2.1. Create an EB tresos Project from Scratch

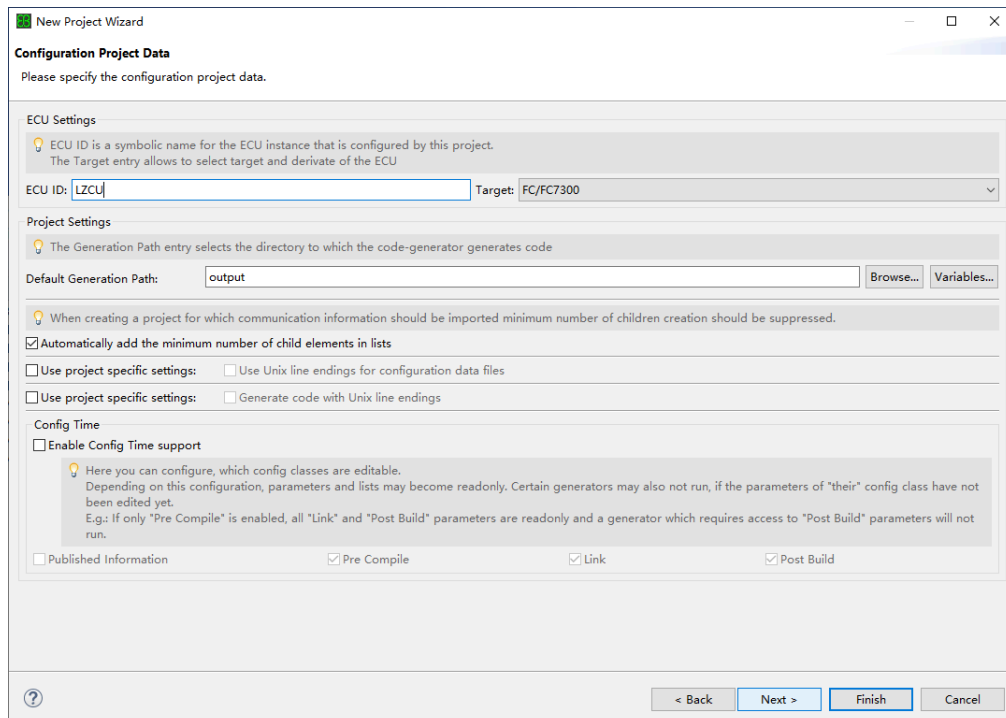
1. Open EB tresos, click **File > New > Configuration Project**.



2. This operation will open the New Project Wizard. Enter the **Project name** and select the project **Location**.



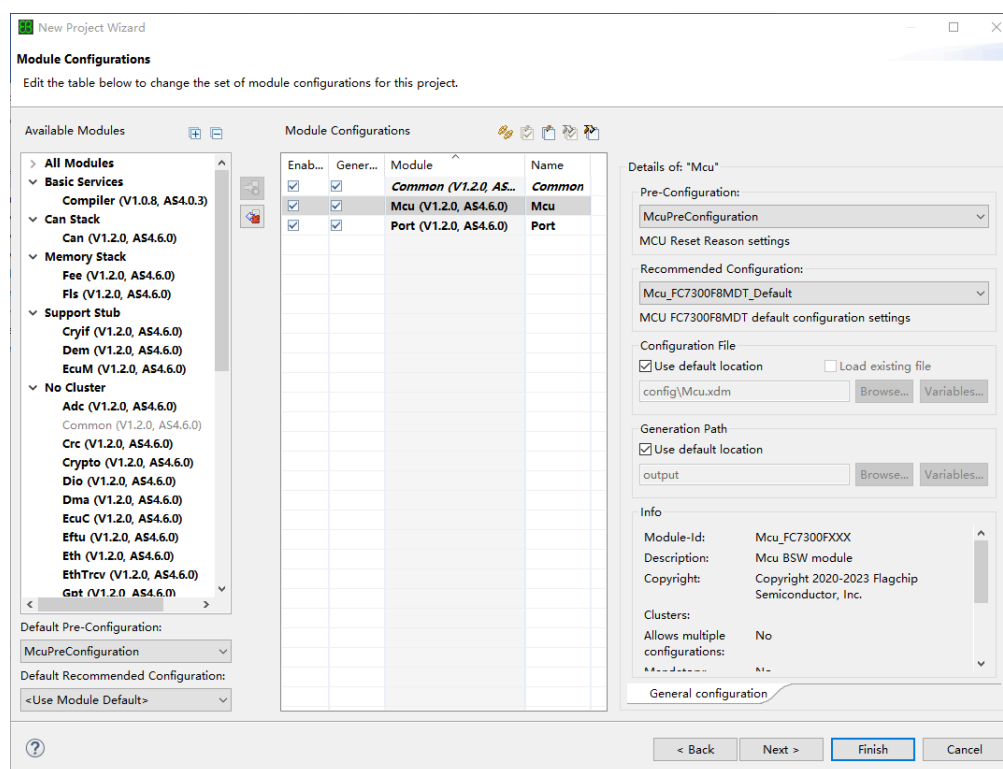
3. In the **Configuration Project Data** tab, the following options will be needed to config.
 - In the **ECU ID** field, Specify a name for the ECU instance.
 - In the **Target** list, select FC/FC7300 target.
 - Check the **Automatically add the minimum number of child elements in lists** option.
 - Click the **Next** button.



Notification

In the **Target** list, you may find two targets for FC7300, which are FC/FC7300 and CORTEXM/FC7300. You could assume FC and CORTEXM as namespace and in most cases they are the same. But if you want to use the BSW provided by Elektrobit, you should select CORTEXM/FC7300.

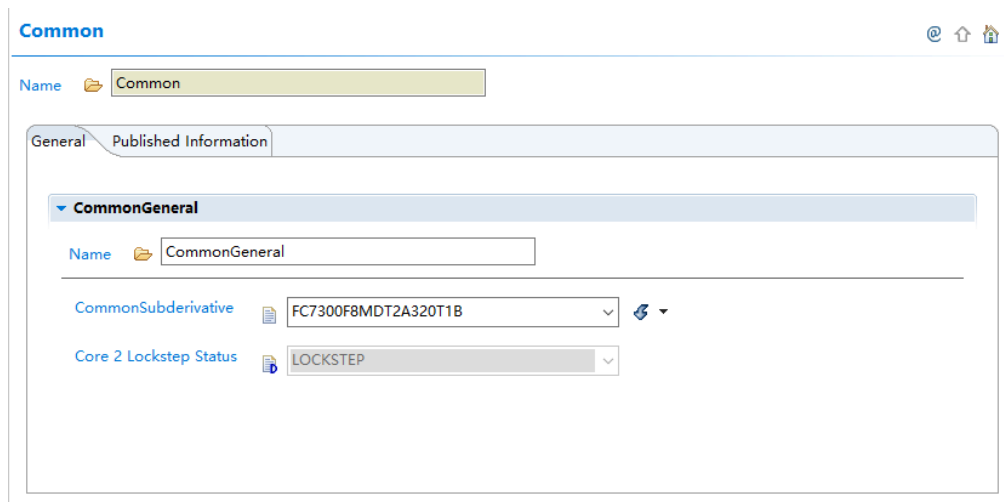
- In the **Module Configuration** tab, you could select the modules you would like to use.
 - The **Common** module is to select the MCU type and is mandatory for configuration.
 - Generally, the **Mcu** and **Port** modules are also necessary for a MCAL project. And for a multi-core project, **EcuC** and **Os** as stub modules are also needed to configure multi-core partitions.
 - Other modules can be selected in need.
 - Most of the MCAL modules provide a **Recommended Configuration** in the hope of accelerating the configuration process. You could select a configuration from the list as the configuration base or you could select None if you want to configure from scratch.



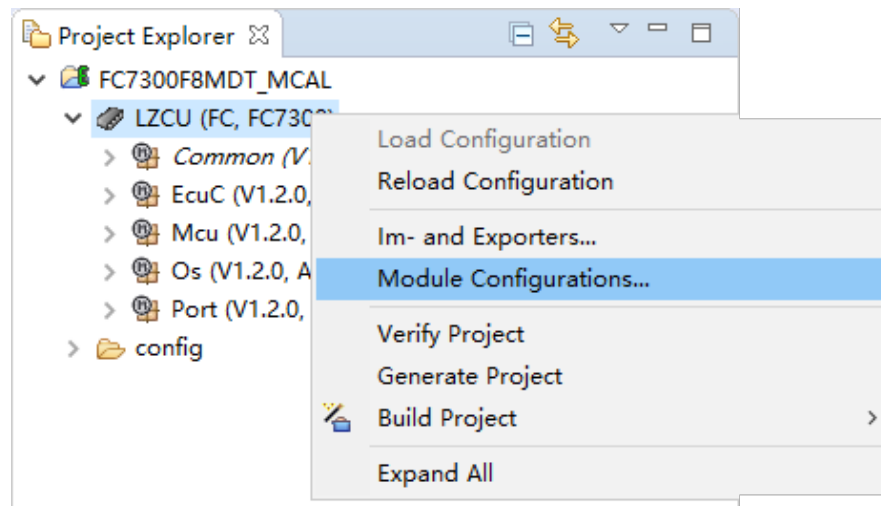
Notification

If you selected **ACM-Base** when installing EB tresos, the **Compiler** module will be added automatically. You should remove this module when configuration since this module is provided by EB and cannot be opened with Flagchip license.

- After create the project, you could change the chip variant in the **Common** module.

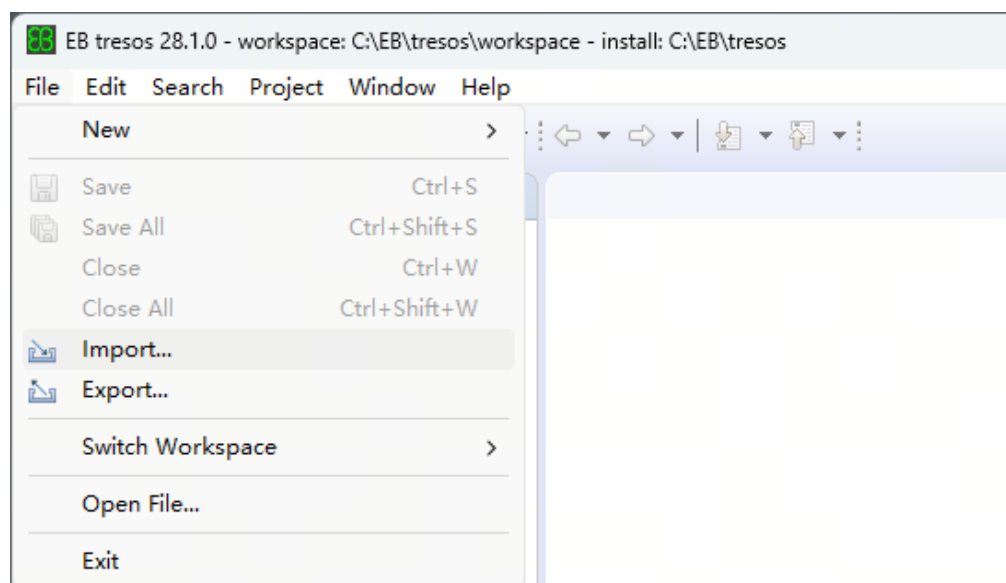


6. If you would like to add/remove modules to the project, you could right click on the project name and select **Module Configurations....** And if you want to generate the configurations, you could select **Generate Project**.

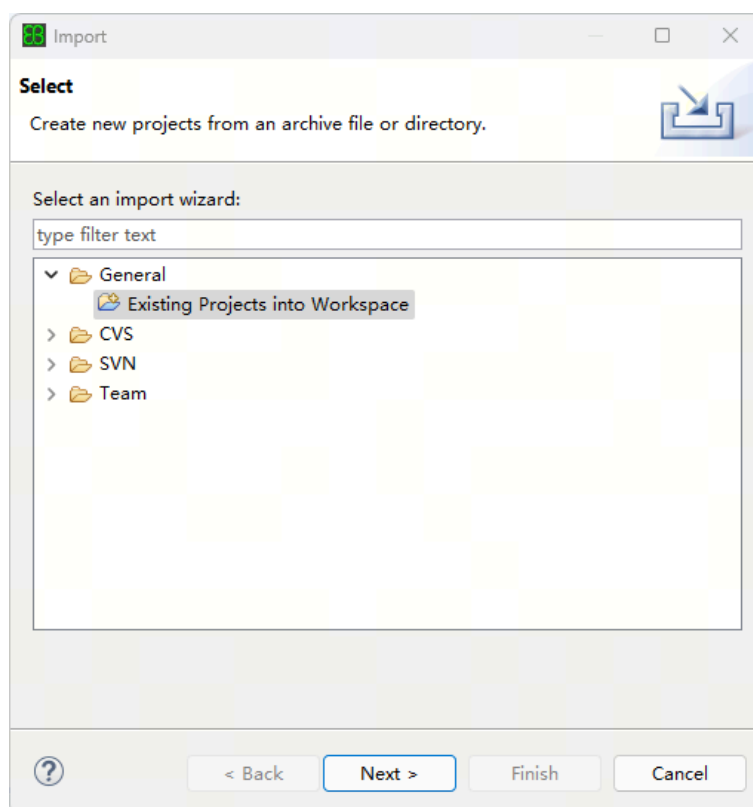


3.2.2. Import an Existing EB tresos Project

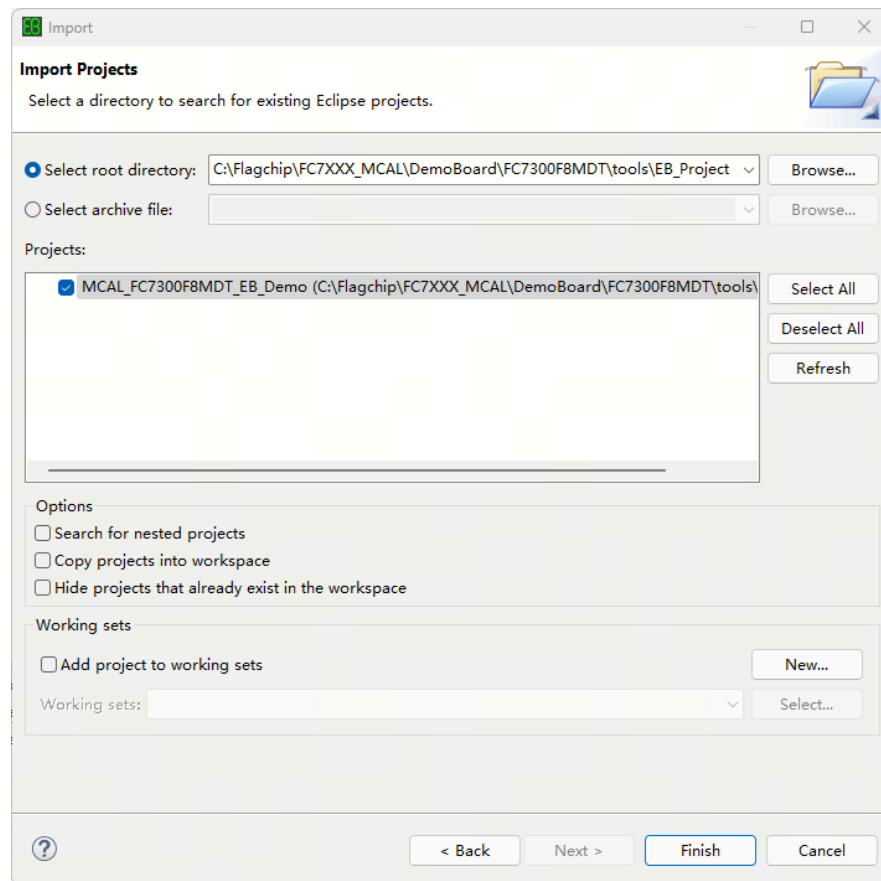
1. Open EB tresos, click **File > Import....**



2. In the EB tresos Import Wizard, select **General** > **Existing Projects into Workspace**.



3. Browse to the folder containing the EB tresos project and select the Project to import. And then click **Finish**.



4. After that, the project is imported to the workspace and you can double click on the project name to modify configurations.

3.3. MCAL Project Integration

3.3.1. Integration of Configurations with Static Code

To integrate an AutoSAR project, you will need the MCAL static code, MCAL configuration, AutoSAR OS, BSW static code, BSW configuration, User app code, Startup code with a link script to link all these code together.

We provide some integrated demos in the DemoBoard folder. Since we do not provide BSW, we use some stub modules to serve as the BSW and OS. The user could integrate their own project refer to the demos we provided.

FC7XXX MCAL package has integration manual and user manual in each MCAL module plugin, integrator could check the user manual to get familiar with module design, module function and module item configuration. Also, integrator shall follow integration manual step carefully to add each MCAL module to his own ECU project. The contents of an integration manual is shown in Figure 5.

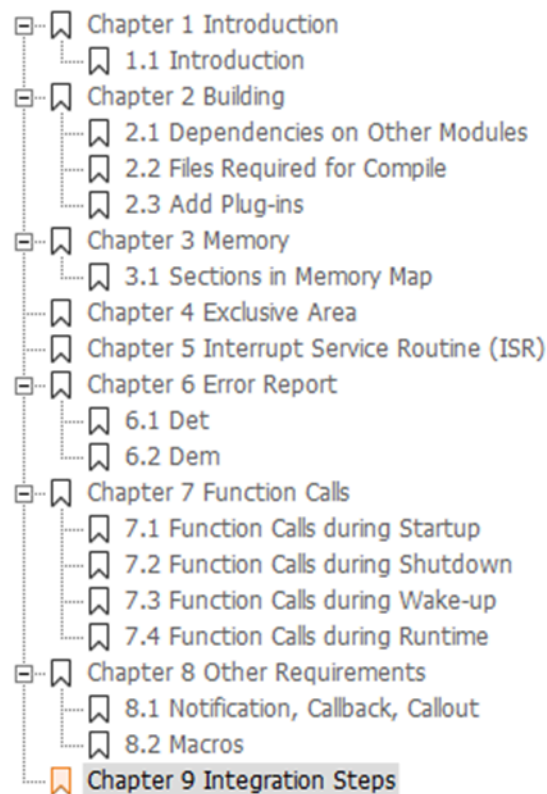


Figure 5: Integration Manual Guidance

MCAL package product is delivered in source code.

Below compiler integration is just for MCAL function demo, user shall use function safety compliant compiler. Which shall meet compiler function safety requirements:

1. compiler meeting TCL 3, ASIL-D scenario.
2. compiler build-in basic C library, shall be developed by ISO 26262 ASIL D requirement.

The compilers we recommend including: FuSa version of IAR Embedded Workbench for ARM, GreenHills Compiler for ARM, HighTec ARM Development Platform, Tasking VX-toolset for ARM.

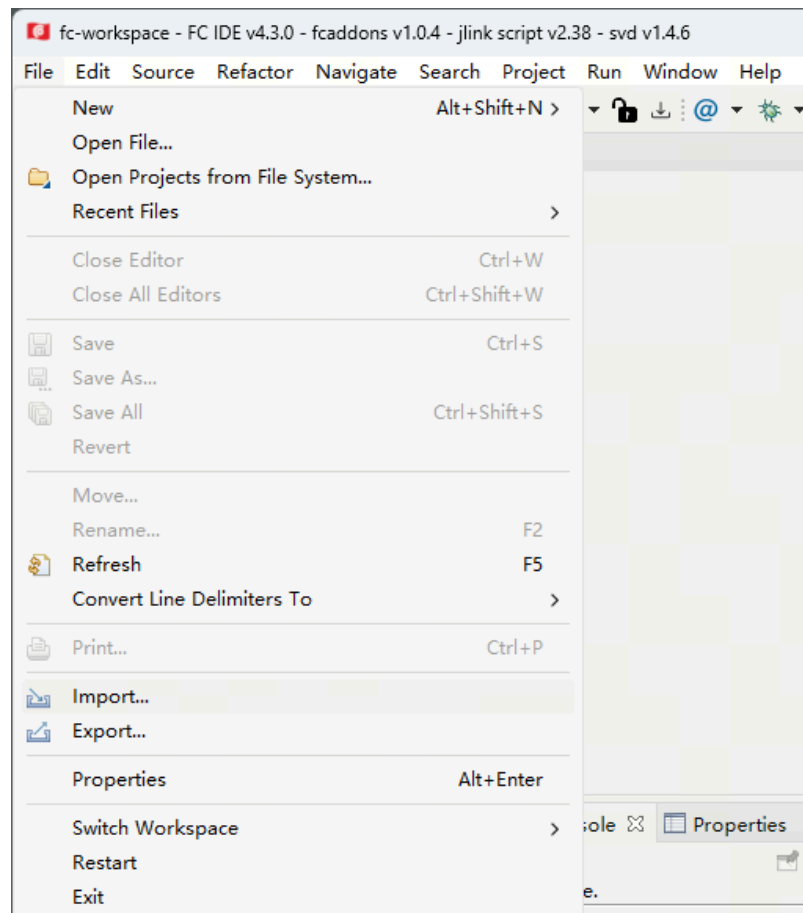
FC_IDE provided by Flagchip uses arm-gcc as code compiler which is not compliant for function safety. User can only use the compiler for projects which do not have functional safety requirements.

3.3.2. Build and Debug

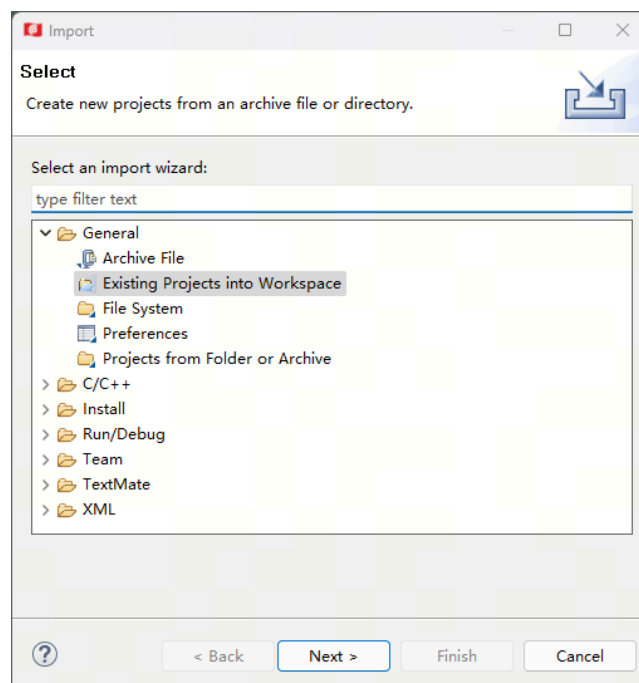
3.3.2.1. Build and Debug with Flagchip FC_IDE

The demo project of FC_IDE combine with GNU compiler can be used as following steps:

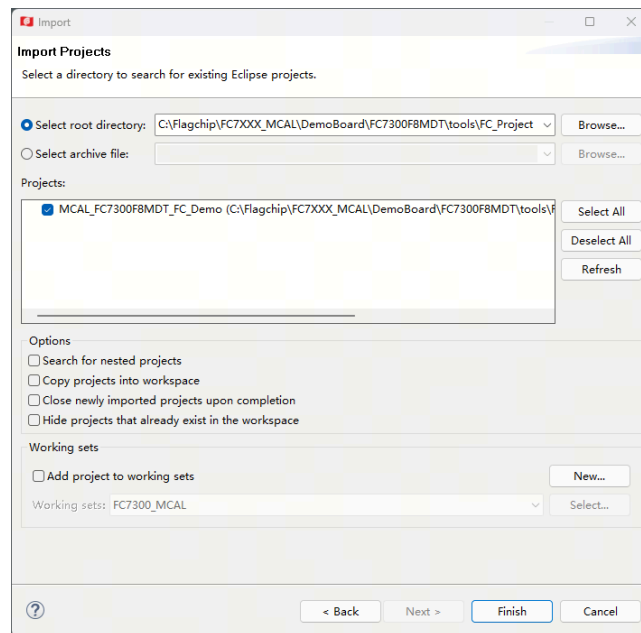
1. Click **File > Import**.



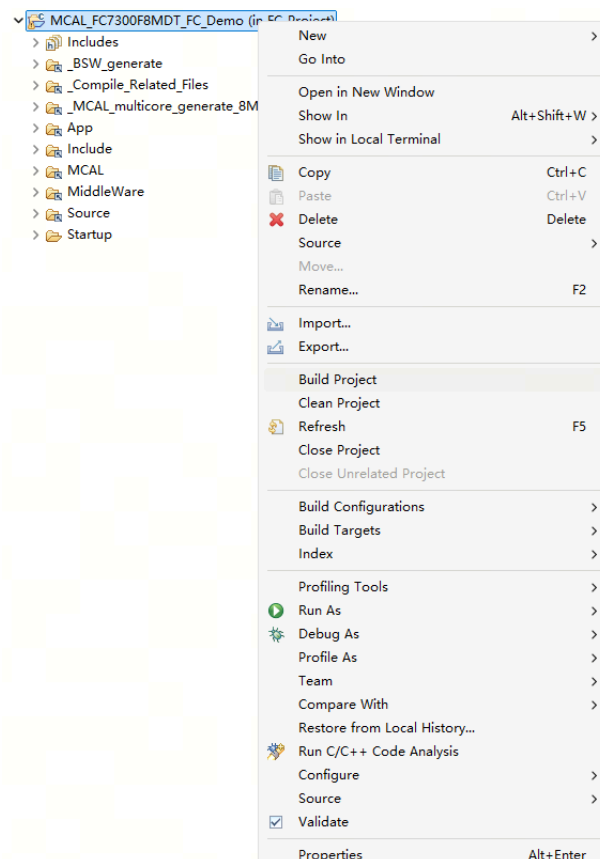
2. In the Import Wizard, select **Existing Projects into Workspace** and then click **Next**.



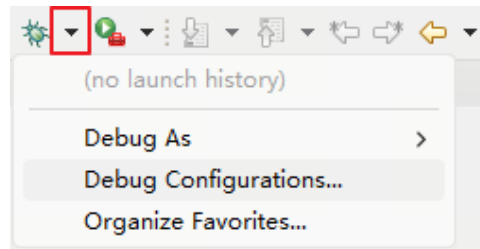
3. Click **Browse** and select DemoBoard/<Chip_Variant>/tools/FC_Project folder. The <Chip_Variant> means the chip you want to use, e.g. FC7300F8MDT.



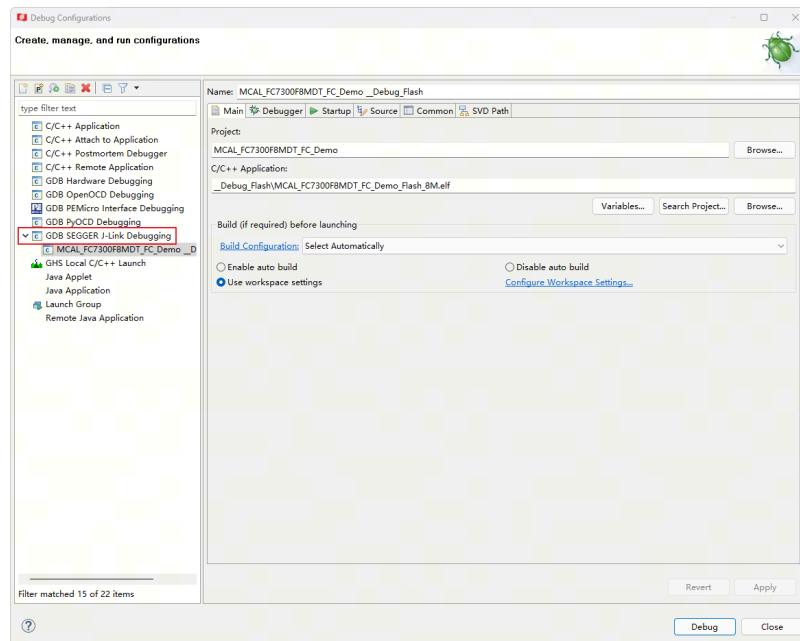
4. Select the existing project and click **Finish**.
5. Right click on the project name and click **Build Project**. If no compiler error occurred, you will get the elf, hex and map file after compilation.



6. Power on the Demo Board and connect the debugger to it.
7. Select the drop-down list arrow next to the debug icon and select **Debug Configurations**.



8. In the Debug Configurations wizard, create a new debug profile. If you are using J-Link, double click the **GDB SEGGER J-Link Debugging**.

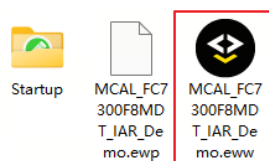


9. Click **Debug** and the program will start to debug.

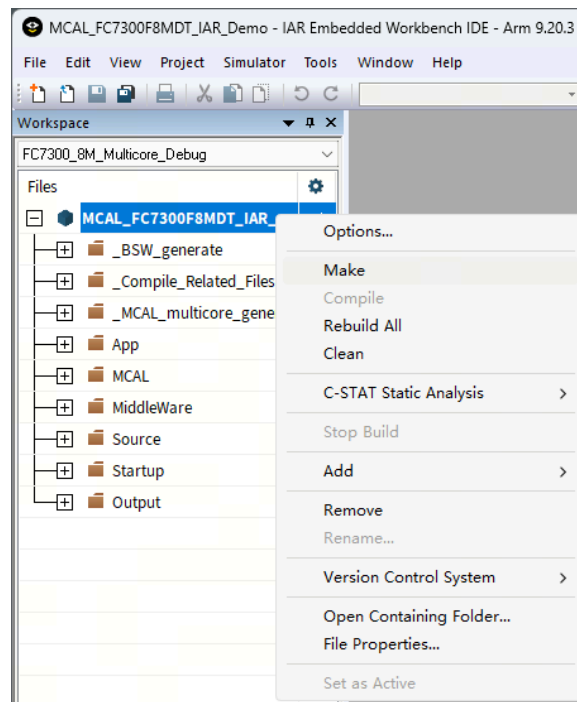
3.3.2.2. Build and Debug with IAR Embedded Workbench

In order to debug with IAR, you will need to download and apply the IAR patch from our [Tools](#) site. Then you could follow the following steps to build and debug with IAR.

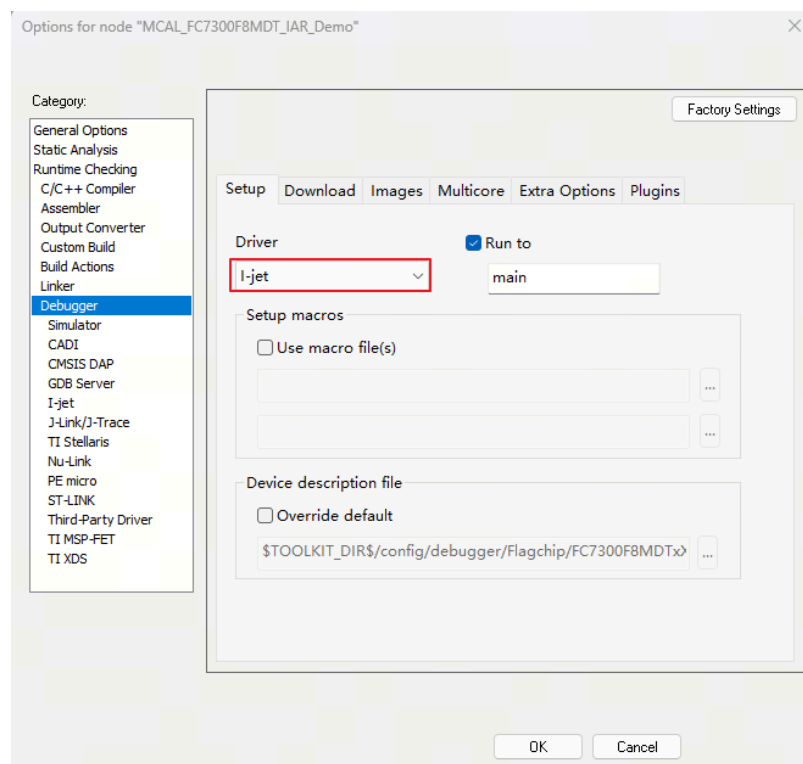
1. Open the IAR workspace file with IAR embedded Workbench.



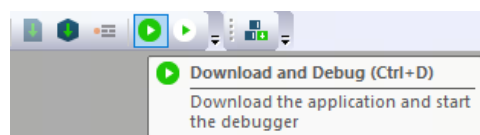
2. Right click on the project name and select **Make**.



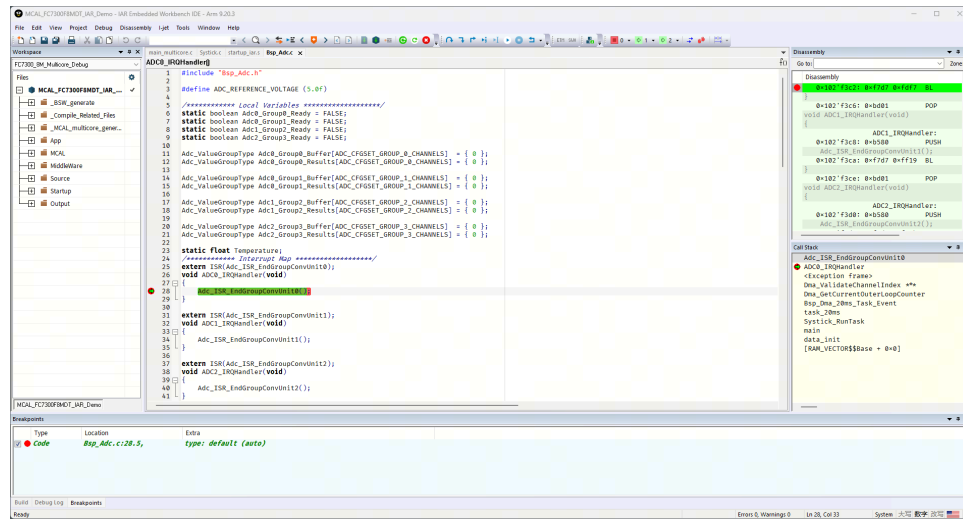
- After the build process completed, right click on the project name and select **Options...**, and you will see the **Options** dialog. In the dialog, select the **Debugger** tab and choose **I-jet**, **J-Link/J-Trace** or **PE micro** as your debug driver. Then click **OK**.



- Click the **Download and Debug** button on the toolbar.



- After that, the program will start to debug and debug window will be displayed.



Notification

IAR embedded workbench supports multi-core debug only when you uses I-jet.

Chapter 4. Extra Feature beyond AUTOSAR R20-11

FC7XXX MCAL module provide extra feature that in not defined in AUTOSAR R20-11. These features are required by customer specific application requirement.

Detailed features are shown in Table 4.

Table 4: MCAL Added Features

Module	Feature	Description
ADC	Channel Delay	Each channel could configure a delay when the ADC instance is triggered.
SPI	SPI Slave mode	Slave mode is supported for asynchronous transmission with one job per sequence and one channel per job.
ETH	ETH MII read write via clause 45 protocol	The AUTOSAR only defined read and write interfaces via MII clause 22 protocol, but some ethernet transceivers (such as JL3113) are only accessed via MII clause 45 protocol. Therefore, the additional interfaces are implemented.
CAN	Split FIFO Hrh	Split each FIFO filter into one Hardware Object
PWM	FTU global time base	FTU GTB feature, start multi FTU instance counter at same time
ICU	IcuGetInputLevel	read channel real time input level of relative HW channel
	FTU global time base	FTU GTB feature, start multi FTU instance counter at same time
GPT	FTU global time base	FTU GTB feature, start multi FTU instance counter at same time
WDG	WDG direct service	The watchdog module shall provide the WdgenableDirectService switch to select whether the watchdog servicing routine is implemented as an interrupt routine (by using a GPT driver callback) or directly operated by the user (by call Wdg_Service).
Fee	Foreign block	Foreign block is used to share data block between bootloader project and application project while both projects use the same flash address region.
Fls	ECC handler	New API is added for software integrator to handling flash ECC error during software running time.

Detailed APIs and Configuration items added in each feature are shown in Table 5.

Table 5: MCAL Added APIs and Configuration Items

Module	Feature	Added API	Added Configuration Item
ADC	Channel Delay	No new APIs	add container: AdcChannelDelay
SPI	Slave mode support	No new APIs	add node: SpiHwUnitMode
ETH	ETH MII read write via clause 45 protocol	Eth_WriteMmd Eth_ReadMmd	add node: EthCtrlEnableMmd

Module	Feature	Added API	Added Configuration Item
CAN	Split FIFO Hrh	N/A	N/A
PWM	FTU global time base	Pwm_StartGlobalTime Pwm_StopGlobalTime	N/A
ICU	IcuGetInputLevel	Icu_GetInputLevel	N/A
	FTU global time base	Icu_StartGlobalTime Icu_StopGlobalTime	N/A
GPT	FTU global time base	Gpt_StartGlobalTime Gpt_StopGlobalTime	N/A
WDG	WDG direct service	Wdg_Service	add container: WdgEnableDirectService
Fee	Foreign block	N/A	add node: 1. Fee Swap Foreign Blocks Support 2. Fee Configuration Assignment 3. Fee Maximum Number of Blocks 4. Fee Block Assignment
Fls	ECC handler	Fls_ECC_Handler	N/A

Chapter 5. ECU System Environment

5.1. MCAL Version and Variant

MCAL apply to AUTOSAR R20-11. the BSW SW package should support AUTOSAR R20-11. or If the BSW SW package is using other AUTOSAR version. user shall disable the AUTOSAR inter module version check.

User shall configure the IC variant in common module, this configuration would affect the configuration items in each module configuration items, the constrain of the item would change according to variant selection. Use for Flagchip FC7XXX series IC include FC7300F8M, FC7300F4M, FC7240F2M.

5.2. System Safe State

MCU safe states are defined as below:

- Operation as expectation, function and safety mechanisms are operation correct
- MCU is in RESET
- MCU is power off

User shall set the MCU to safe state while critical issue occurs. If the MCU continuously switches between normal operating and reset state, it is not considered a safe state. The system should be implemented a method to monitor such case.

System shall be able to bring itself into a system safe state. If the system continues to reset between the normal operation state and the reset state, it is not a safe state. the system is able shall identify this case and treat it as an unsafe state.

MCAL modules no not support API functions to switch to system safe state. MCAL support return function error, support report diagnostic event error. Integrator shall be able to switch system to safe state if MCAL function return these errors.

MCAL doesn't define a safety state, provides independent functions to customers, so there is no so-called safe state. However, if there are any problems with the execution of all functions, they will return a value to the customer, who will enter the system level safe state.

5.3. ECU System Software Architecture

Integrators shall perform system level DFA analysis, MCAL modules aims to safety level ASIL D. So, integrator performs ASIL decomposition at the system level to ensure if driver module meet ASIL level meet system safety level requirements.

Integrators directly call MCU API functions at ECU startup stage, MCU provide clock initialize, reset source check, ram initialize API. This function is called normally in EcuM callback function.

Integrators use AUTOSAR standard BSW module communication interface module to communicate with ENET, CAN, LIN, SPI, According to AUTOSAR definition, the upper service layer is communication interface module. Integrator shall integrate this module to call communication Driver modules.

Integrators shall perform E2E protection on data transmission for communication drivers (ENET, SPI, LIN, CAN) usage. E2E protect include the method of Rolling counter, checksum and CRC,

Integrator shall calculate the E2E value and transmit it together with actual transmit payload. The receiver shall calculate again and compare with the received E2E value.

Integrators shall use NVM to manage the ECU NVM block. Shall use Memif to communicate with FEE, and FEE shall call the FLS module. Then integrate NVM Memif FEE and FLS together to read and write NVM data.

Integrators shall perform CRC check for NVM data. The FLS and FEE module are used to read and write NVM data. For data integrity, Integrator shall add CRC data together with FEE data. Write them to NVM together, while reading data, calculate CRC again and compare with the stored CRC.

This mechanism executed in NVM module in AUTOSAR requirement.

Integrators use AUTOSAR standard BSW module WdgM and Wdgif to communicate with Wdg driver. User shall integrate these modules together to achieve task alive, transition sequence... WdgM module calls function of Wdg by Wdgif.

Watchdog timer requires periodic execution of a watchdog servicing operation, otherwise will reset system. Integrator shall feed the watchdog in preconfigured cyclic period.

Integrators use AUTOSAR standard BSW module HwIOAbs module to communicate with PORT, DIO, ADC, PWM, ICU According to AUTOSAR definition, the upper service layer is HwIOAbs module. Integrator shall integrate this module to call the IO Driver modules.

Integrators use RTE interface to directly call complex driver API (DMA, IIC, TRGSEL, OCU, MAILBOX, MSC, SENT, UART) DMA, IIC, TRGSEL, OCU, MAILBOX, MSC, SENT, UART are the specific modules that not defined by AUTOSAR, FLAGCHIP developed the modules to cover MCU extra peripheral. Application shall call the peripheral directly by RTE layer.

Integrators shall consider the DMA data address access range. The access of the DMA bus master to memory shall be restricted to the memory ranges involved in DMA transfers. The restriction concerns source and destination memory ranges. Before assign DMA data transport, user shall check the DAM data access setting range.

MCAL module reports running error by DET and DEM errors. Integrator shall use DET module for development stage. Integrator shall use DEM to report MCAL running error, for some Severe item, should save it in NVM.

Integrators shall detect and handle interrupt flooding, e.g. by means of control flow monitoring. The ISR handler function callback shall always reset the interrupt status flag as soon as possible.

Integrators shall provide an appropriate response for exceptions. Exception shall be captured and handled with proper solution (even if no register protection is used).

Integrators shall provide the same ASIL level notification or callout handlers to MCAL module. MCAL module would provide callout and notification handler functions for user to implement relative response or notify upper layer application function module.

5.4. Multicore Considerations

The FC7xxx series MCUs have single-core products and multi-core products. For multi-core products, the MCAL can be used as a multi-core product or as a single-core product.

When the MCAL is used as a single-core product, we treat each core as a separate product. You shall build the project for each core and burn into the specific location of the Flash. You should take care not to use the conflict resources and design an inter-processor communication mechanism.

When the MCAL is used as a multi-core product, we treat the MCU as a whole. All cores could share the same code base. You could allocate the resources in configurations of the MCAL. On development phase, you could enable the DET and the MCAL could check the resource confliction and report a DET error. This section focuses on the multi-core usage.

In order to configure the multi-core feature of MCAL, the EcuC and the Os modules shall be added to the EB project. These two modules are provided as stub modules in the MCAL and only serve as configuration interfaces of other modules. When the MCAL is integrated into BSW modules, the stub modules shall be substituted by the corresponding modules in the BSW.

The user shall define EcuC Cores (Figure 6a) and EcuC Partitions (Figure 6b) in the EcuC Modules.

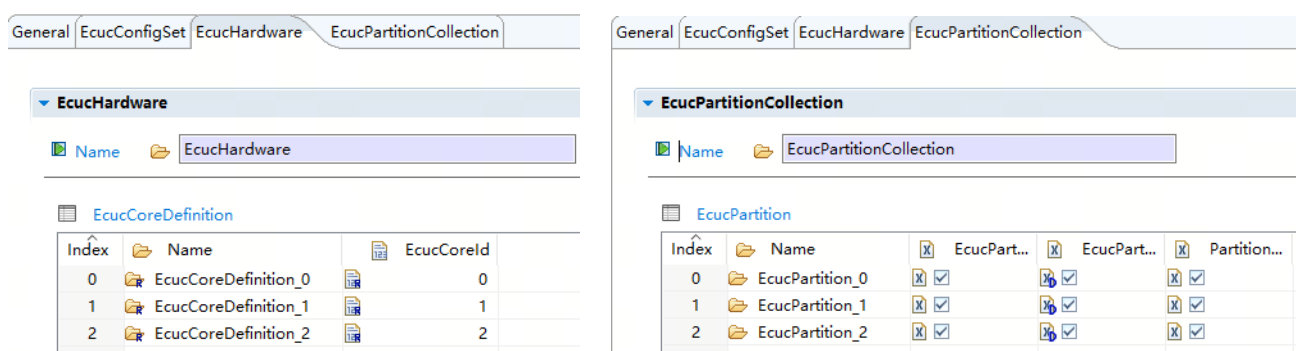


Figure 6: EcuC Configurations

And then the user shall correlate the core with the corresponding partition in the Os module. As shown in Figure 7.

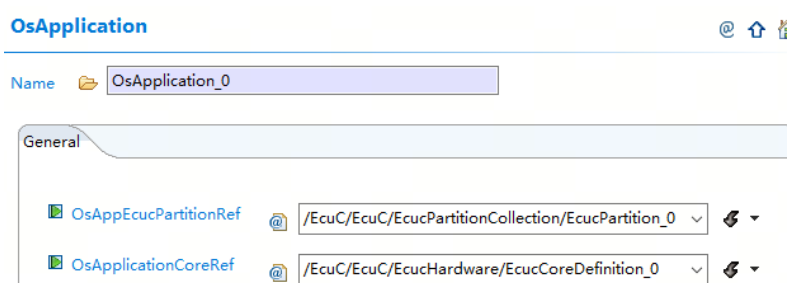


Figure 7: Os Application Configurations

Most modules have a multi-core support option. After enabling the option, the user shall refer to the EcuC partitions defined in Figure 6b from the module.

5.4.1. Multicore Startup Procedure

A module is initialized in the context of core, which means each core shall call its initialization functions independently. However, some modules shall be called in the master core, which is core 0, before activating the slave cores, such as Mcu, Port, Dma, TrgSel and Eftu. This is because these modules are served as basic infrastructures and other modules may use the resources provided by these modules. Besides, the initialization function of Mcu and Dma shall also be called in the slave cores to initialize some global states for the separate core.

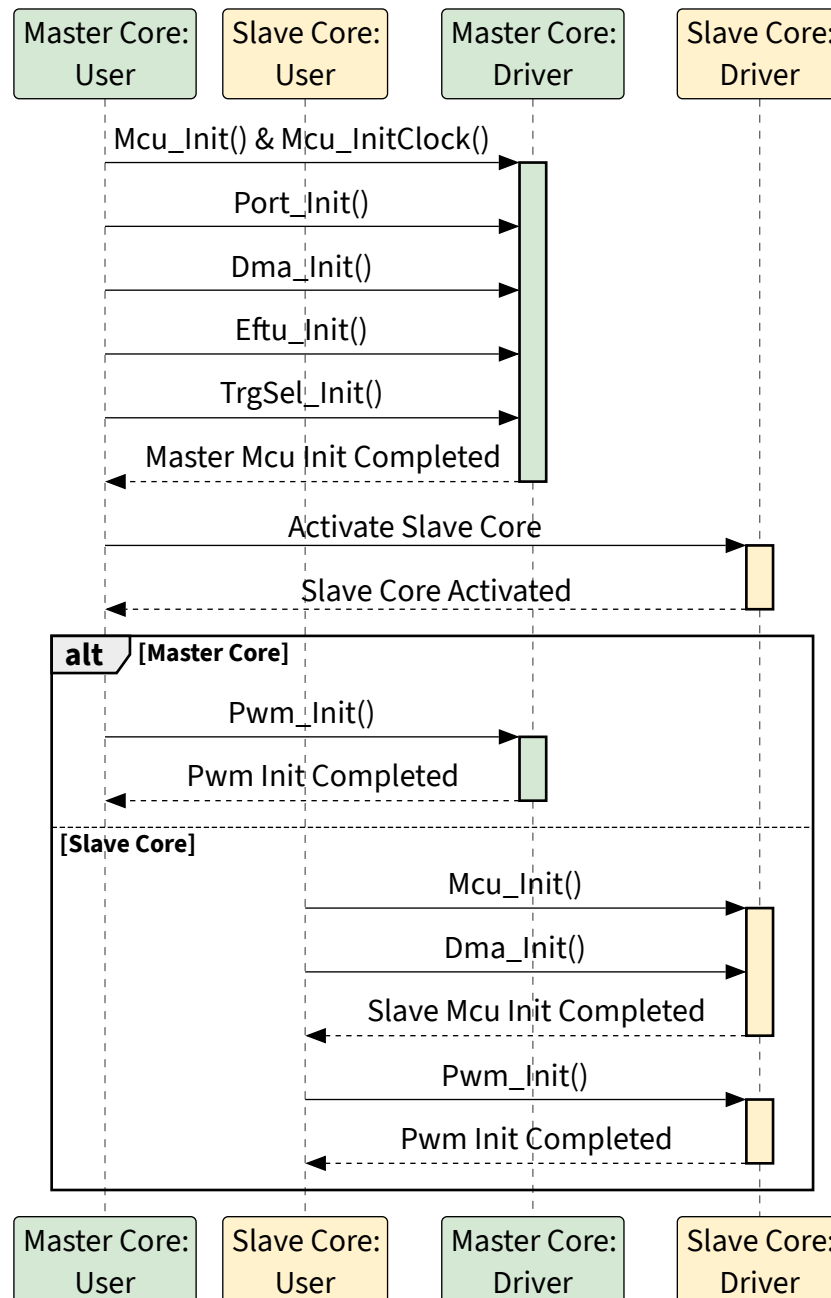


Figure 8: Multicore Initialization

Figure 8 shows the general initialization procedure of the Pwm driver in a multicore configuration.

5.4.2. Multicore De-Initialization

Similar to the initialization process, the de-initialization process should also be done in the context of core. Each core should shall call its initialization functions independently, except the basic infra-

structure modules, which could be called only when all of the dependant modules are de-initialized. And this shall be guaranteed by the user who uses these modules.

5.4.3. Multicore Interrupts

Hardware resources assigned to a CPU must have their interrupts routed to the same CPU. This must be ensured by MCAL users.

Interrupts of resources allocated to a core if serviced by different cores will require complex implementation of multiple spin locks and possibly reduce the efficiency of the software.

The interrupt service routines provided by modules are capable to be executed on several CPU cores in parallel.

The interrupt request could be enabled by NVIC registers in the core. Each core shall enable its IRQs in the NVIC registers. The MCU also provides the `SCM_INT_ROUTERn` registers in the SCM module which could be used to router interrupts to specific cores. Besides, the `DWP` field of `PORT_PCRn` registers will also affect the interrupt router for the Port interrupts. For more information, please refer to the reference manual.

5.4.4. Data Consistency in Multicore System

This MCU does not support the exclusive instructions `LDREX` and `STREX`. When executed, these two instructions will behave the same as normal `LDR` and `STR` instructions. In order to support exclusive access, we provide a hardware module **Mailbox** and you could use the **Mb** module in the MCAL for spinlocks.

This MCU only implements the L1 cache and thus does not support the `S` bit attribute in the MPU region settings. The user shall take care of the memory settings for shared memory.



Notification

Although `LDREX`, `STREX` and `S` bit are not supported, the side effects of still take effect, this includes:

1. If a memory region enables data cache, and also has the `S` bit attribute set, the memory region is still treated as non-cacheable. This behaviour could be changed by setting the `SWIT` bit of the `CM7_CAPR` register.
2. If `LDREX` and `STREX` are executed in a memory region tagged with `shareable`, which means the memory region has the `S` bit set, or the memory region is configured to Strongly-ordered or Device, a Busfault will be reported.

5.4.4.1. Multicore Shared Data Consistency

The FC7xxx series MCUs have multiple memory masters, including the multiple Cortex-M7 cores, the Flexcore, the DMAs, the HSM and the Enet. When operating between masters, the user must take data consistency into consideration. The memory type and cache can be changed with the MPU module in the Cortex-M7 core.

The memory region configured as Strongly-ordered or Device is shareable between cores, which means the user does not need to take extra actions to guarantee data consistency. However, these

two types of memory region do not allow unaligned data access, which will result in some inconvenience in usage.

Because of the write buffer in Cortex-M7, which will result in the delayed write of data to memory, the memory region configured as Normal cannot by nature guarantee data consistency. However, the `dsb` instruction could force an immediate write into memory. And thus we could use the `dsb` instruction to guarantee the data consistency for memory regions configured as Normal. That is, if a master needs to write a variable to a shared region configured as Normal memory, it could add a `dsb` instruction before it notifies the other masters which need to receive the variable. The notification mechanism includes sending an inter-processor interrupt with Mailbox peripheral or start the DMA transfer, etc.

It's not recommended to enable cache in shared data and if cache is enabled in shared data, the user shall take care of the cache clear and invalidation besides the write buffer.

The ITCM and DTCM are tight coupled memory of the core and other cores/masters cannot access the memory through the normal address. If needed, other masters could access the ITCM/DTCM of specific core through the backdoor address while this would be much slower than accessed by the belonged core itself. When accessed through the normal address, cache cannot be enabled on ITCM/DTCM. But when accessed through backdoor address, cache can be enabled to accelerate data access.

5.4.4.2. Data Consistency when Cache Enabled

Cache is accessed by cache line, and thus it will sometimes result in data consistency problem even there is no data shared in the same cache line when the cache is set to write back.

The problem is that the cache line of Cortex-M7 is 8 words, or 32 bytes. Figure 9 to Figure 12 shows an example of data conflict when core 0 and core 1 access data in the same cache line while the memory is set to write back. The lines marked with means variables owned by core 0, and the lines marked with means variables owned by core 1. No data are shared across cores. However, when core 0 modifies a variable in the cache line, only the variable in the cache will be changed, and the cache line will be marked as “dirty”, as shown in Figure 9.

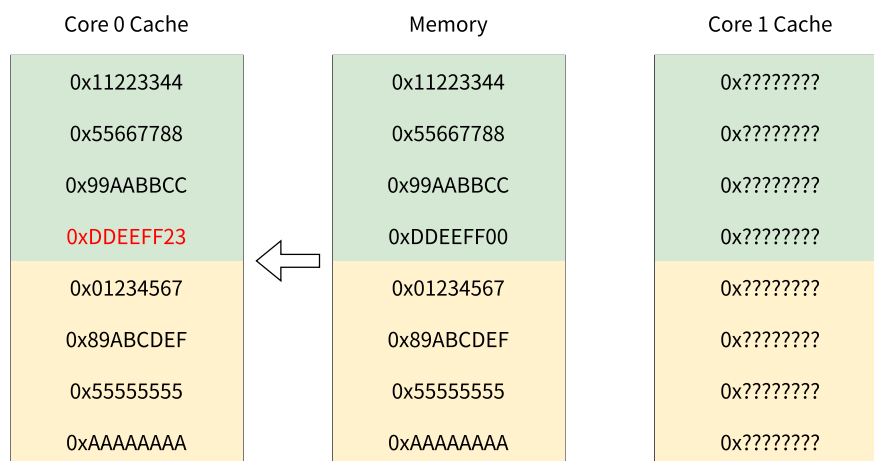


Figure 9: Data Modified by Core0

And when core 1 modifies a variable in this memory region, it will fill in the cache line with the data in the memory and it has no knowledge of the data changement in the cache of core 0, as shown in Figure 10.

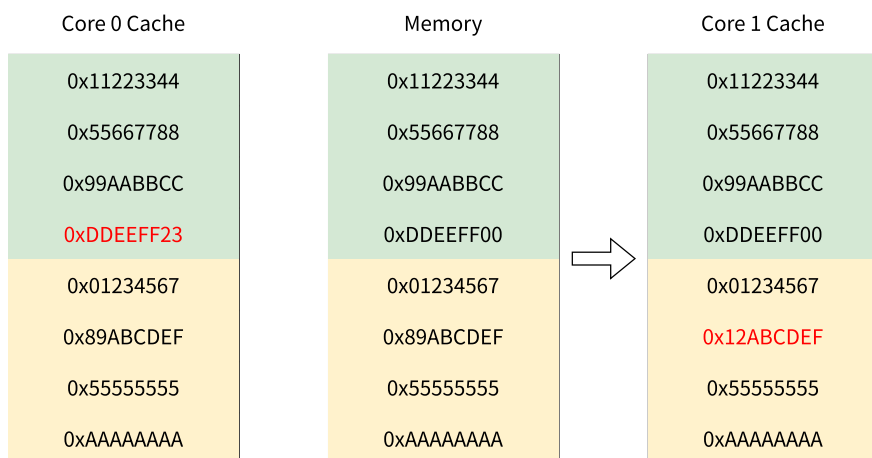


Figure 10: Data Modified by Core1

And when cache in core 0 is evicted, the cache will be cleared and the data will write back to memory, by cache line, as shown in Figure 11.

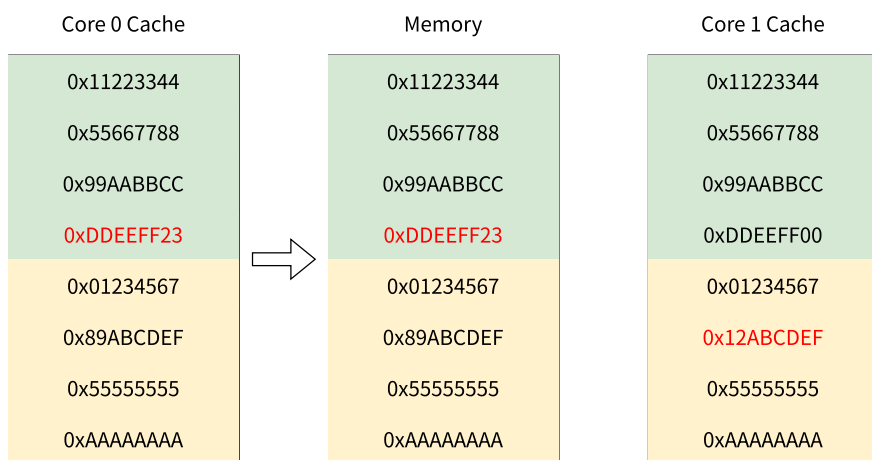


Figure 11: Data Write Back by Core0

And when cache in core 1 is evicted, the cache will also be cleared and the data will write back to memory, by cache line, as shown in Figure 12.

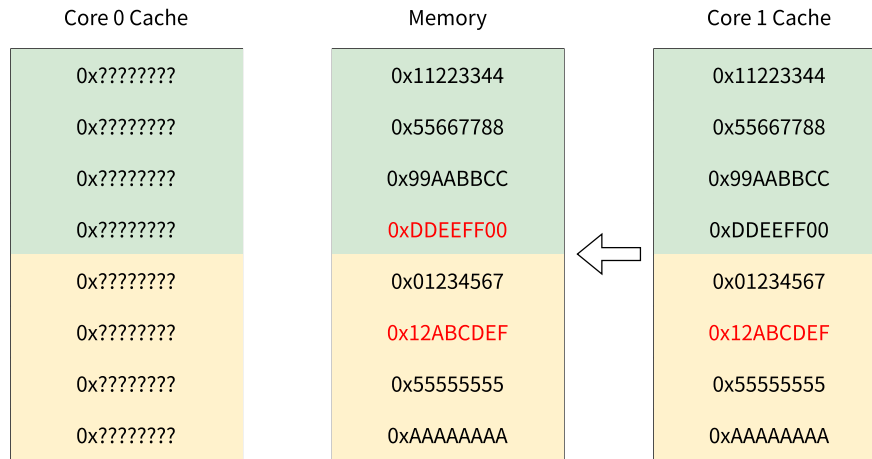


Figure 12: Data Write Back by Core1

We could see that the 4th word of the memory is changed unexpectedly by the write back process of core 1, which results in the data inconsistency of core 0.

This story tells us that if the memory in a cache line will be modified by multiple cores, even when the data are not shared between cores, there will be data consistency problems when the cache policy of the memory region is write back. This problem could be avoided by using the write through cache policy, which will write the data to cache and memory alongside.

The data sections for variables used in the MCAL can be grouped into `.mcal_data`, `.mcal_bss`, `.mcal_nocacheable_data`, `mcal_nocacheable_bss`, `mcal_shareable_data` and `mcal_shareable_bss`. Data cache with write through policy can be enabled on `.mcal_data` and `.mcal_bss` sections. And data cache cannot be enabled on the other section groups.

Chapter 6. AUTOSAR Ecosystem

For FC7XXX series software development, Flagchip cooperates with AUTOSAR system software partners for ECU whole software system Integration. The ecosystem includes AUTOSAR basic software package and MCU safety lib package.



Notification

FC7XXX MCAL is just driver part of MCU that arranged in AUTOSAR standard, doesn't support MCU ASIL-D safety mechanism. If integrator want to fully implement MCU ASIL-D safety mechanism, should purchase and integrate safety lib part. Or Integrator can develop software solution of safety mechanism independently, Flagchip would provide safety demo in SDK driver mode.

6.1. AUTOSAR BASIC SW Package

6.1.1. MICROSAR Classic Vector SLP4

Supported compiler: Green Hills, Version: 2012

Supported AUTOSAR version: AUTOSAR 4.x and Rxx-11

MICROSAR Classic Vector SLP4 does not implement OEM specific requirements.

The individual AUTOSAR basic software modules are grouped into MICROSAR Classic products that consists of related functions. The configuration of the MICROSAR Classic basic software with DaVinci Configurator Classic is simple, user-friendly, and consistent. The included command-line-based generators create optimized code for your ECU based on the configuration.

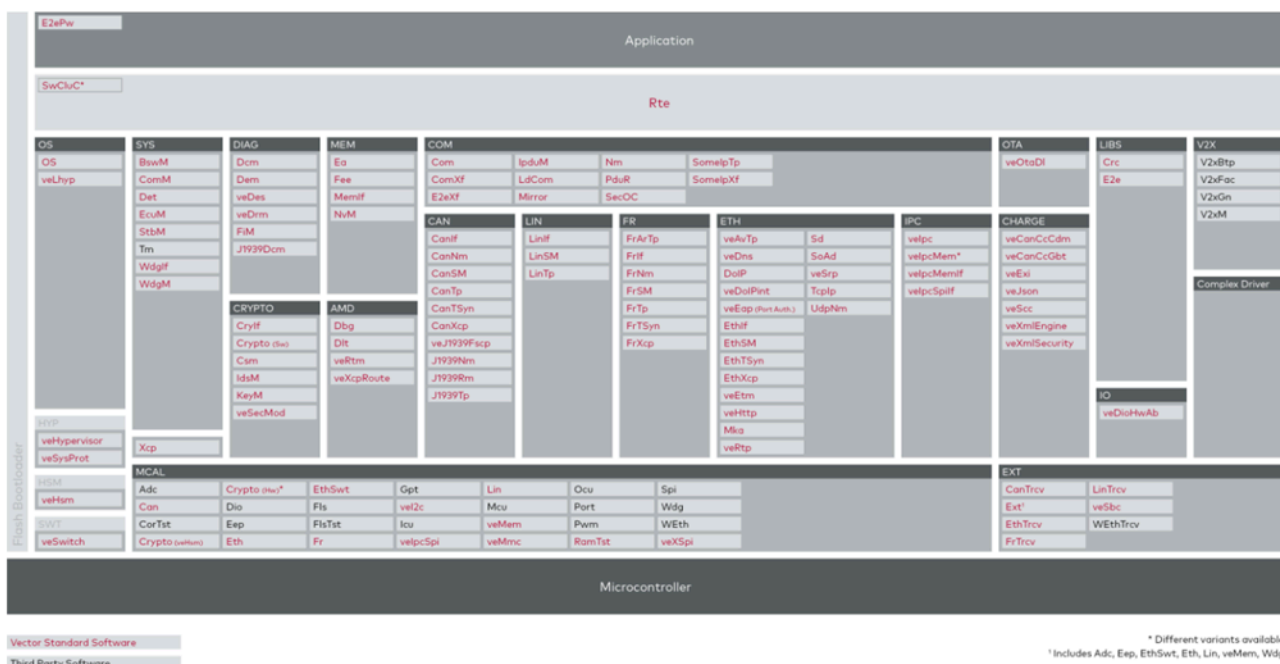


Figure 13: MICROSAR Classic Vector SLP4 Modules

6.1.2. INTEWORK-EAS-CP

Supported compiler: IAR/GreenHills

Supported AUTOSAR version: AUTOSAR 4.2.2/R22-11

Provides a complete AUTOSAR tool chain, compatible with a variety of mainstream data formats in the industry, such as DBC, LDF, PDX, ODX, ARXML, etc., and supports seamless integration with third-party AUTOSAR tool chains.

Provides more than 50 standard modules, embedded source code delivery, and modules can be flexibly combined and purchased.

Development process and products have obtained functional safety ASIL-D dual certificates, provided complete functional safety documents and services, and helped customers more easily achieve functional safety-related requirements.

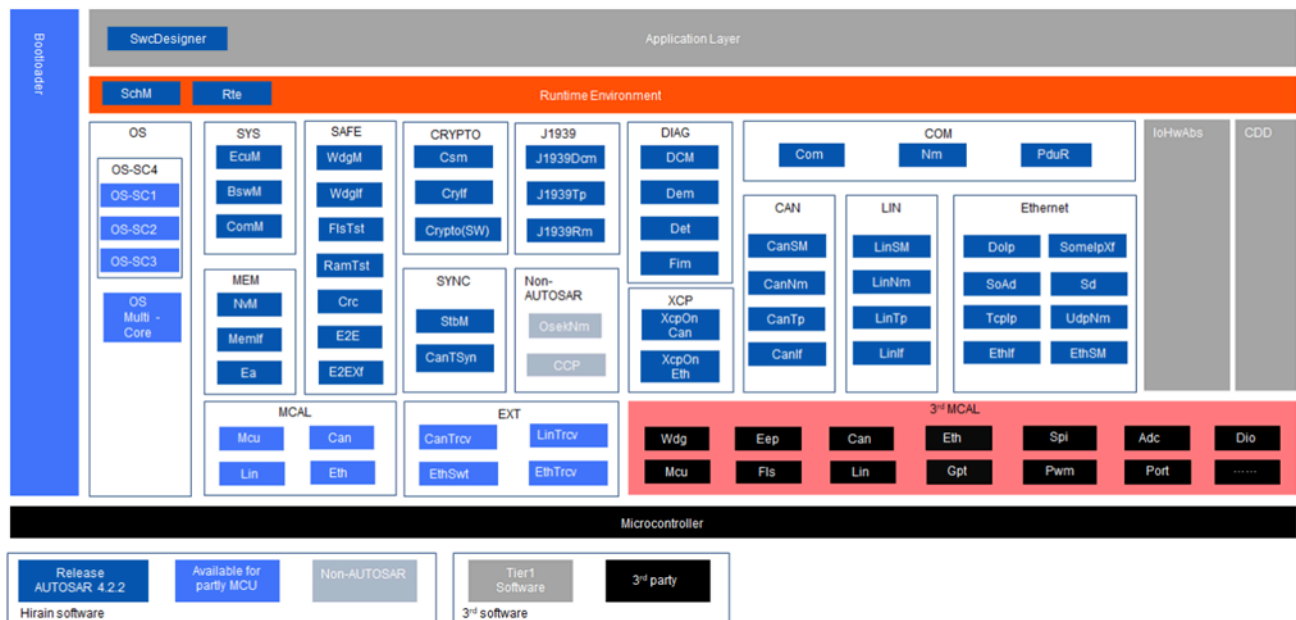


Figure 14: Hirain INTEWORK-EAS-CP Modules

6.1.3. ORIENTAIS Classic AUTOSAR (CECT)

Supported compiler: IAR/GreenHills

Supported AUTOSAR version: AUTOSAR 4.x and Rxx-11

Complete Classic AUTOSAR integrated development environment, including SWC design tools, BSW basic software configuration tools, RTE configuration and code generation tools.

AUTOSAR Safety OS products have passed the German Rhine ISO 26262 functional safety ASIL-D highest safety level product certification.

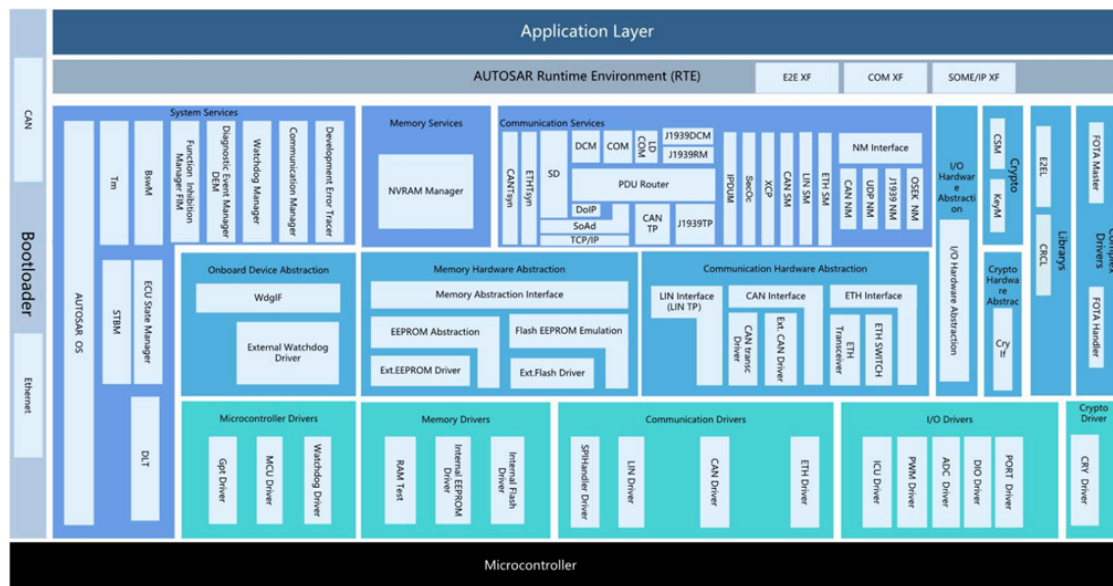


Figure 15: CECT ORIENTAIS Modules

6.1.4. NeuSAR cCore (Neusoft REACH)

Supported compiler: IAR/GreenHills

Supported AUTOSAR version: AUTOSAR 3.1.5/4.2.2/R19-11

NeuSAR cCore is developed based on the AUTOSAR Classic platform standard and independent research and development. It provides core functional modules of controller basic software with high real-time requirements, realizes independent integration of the basic software platform of the whole vehicle, and meets the basic software requirements of mainstream ECU products.

OS, RTE, WDG, E2E and other modules have passed the functional safety ISO26262 ASIL-D certification.

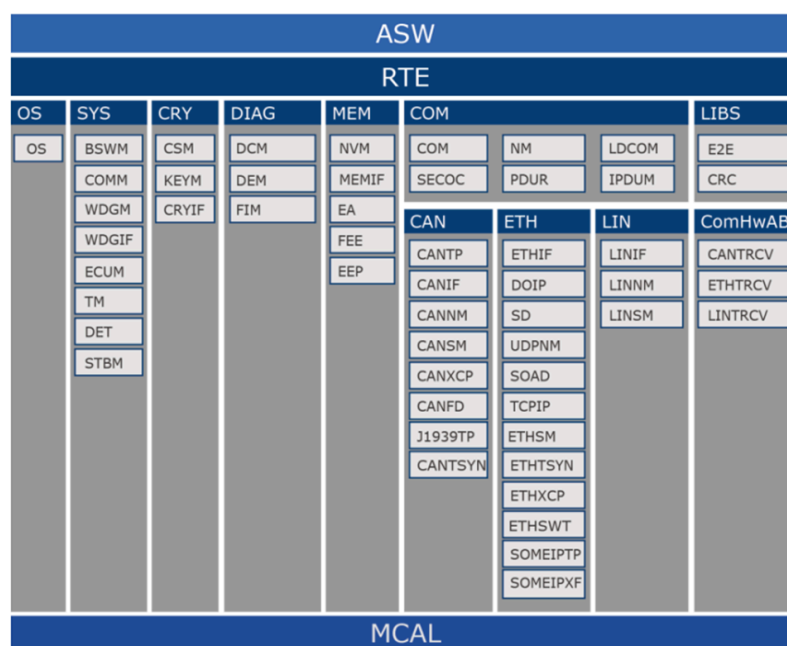


Figure 16: Neusoft NeuSAR Modules

6.2. MCU Safety Library

6.2.1. FC7XXX Safety Frame (ZhiCong)

Supported compiler: IAR/GreenHills

Can be integrated into AUTOSAR as a complex driver.

Can be integrated into non-AUTOSAR software architectures, flexible adaptation.

ISO 26262-5 (2018) Clause 8 introduces two metrics: Single point fault metric and Latent-fault metric.

According to different ASIL level requirements, single-point fault metric and latent fault metric need to reach the corresponding level. For microcontrollers (MCU, hereinafter referred to as MCU), in electronic and electrical systems, they are designed and developed as SEooC (safety element out of context). To meet the above two metric requirements, MCU needs to implement the corresponding safety mechanism.

The safety mechanism can be allocated to hardware and software modules. The Safety Frame safety library of MCU is to implement the safety mechanism allocated to software.

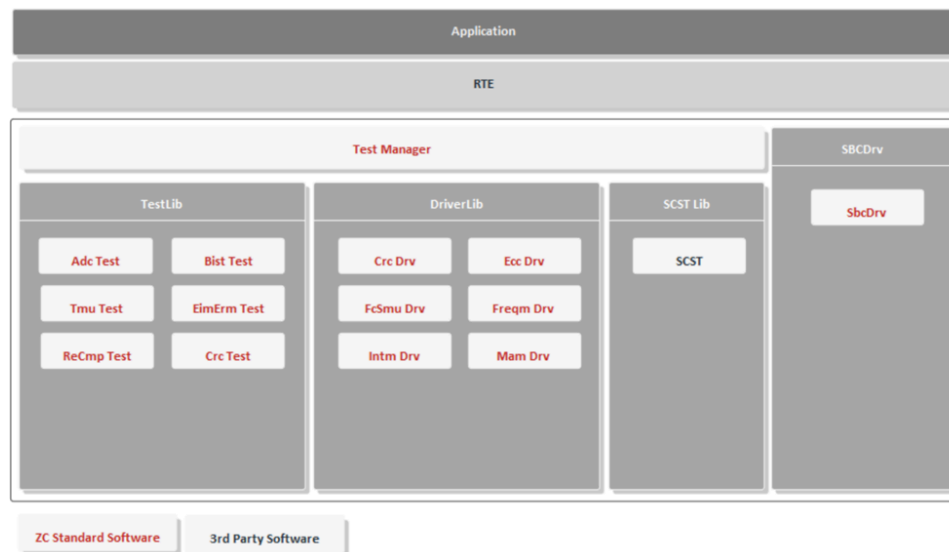


Figure 17: ZhiCong Safety Library Modules

Chapter 7. MCAL safety mechanisms

MCAL driver has internal safety mechanism, both at module configuration stage and static code running stage. For module configuration, below constrain shall be noticed.

Each module has DET check option in the EB Tresos Studio tool and will report module errors by predefined error type. Integrator would use this mechanism for debug errors in development stage and shall disable it in production stage.

Each module support DEM in EB Tresos Studio tool and is designed to report predefined error module errors type to DEM module. Integrator would use this mechanism for MCU hardware error recording.

Each module provides configuration constrains, these constrains may root in hardware internal request. For example, peripheral clock shall be enabled before peripheral executes. Also, some constrains may root in EB Tresos tool code generating limitation.

In module configuration, configurator shall pay attention to the configuration warnings and resolve all the warnings. These warning may cause by parameter value setting out of response range. Or cause by function callback not set but callback feature enabled.

If unreasonable configuration parameters appear in EB Tresos Studio, these parameters shall not be generated, and the tool will generate errors (if code generation is performed). In module configuration, configurator shall pay attention to the code generation errors and resolve all errors. These error reporting mechanism to designed to prevent incompatible hardware resources configuration and prevent situations that conflict with safety requirements.

In static code running stage, assume user finished the dynamic data generation and is calling module API for MCAL integration in whole user application project.

Module would detect conflicts and incorrect calls of interfaces within module initialization stage like scope check, error data check, logical sequence of data processing, process control, etc.

For example, when calling the module API, the data structure used to save the configuration parameters of the module driver and hardware unit is detected at the beginning. If the structure is empty, it will trigger initialization failure and stop the required operation. In this case, the module will report an error.

MCAL module initialization function (<Module>_Init) of the module shall be called first before calling other APIs of the module.

Integrator shall strictly follow API calling sequence request by integration manual and AUTOAR requirement.

Integrator shall strictly configure the module and generate API function used data instance by relative API function is called.

Chapter 8. MCU safety mechanisms

MCU safety development is based on safety assumptions. During the system level development, the safety system developer is required to establish the validity of the MCU assumptions in the context of the specific safety-related system. All relevant MCU assumptions can be identified as follows:

- Assumption (ASR): An assumption that is relevant for functional safety in the specific safety system. It is assumed that the safety system developer fulfils an assumption in the design.
- System needs to check if the assumption does not meet the application requirement

Safety system developer needs to check whether these assumptions are valid for their safety system. In the case an MCU assumption does not meet system requirements, the system developer should initiate a change management activity beginning with impact analysis.

For example, if a specific assumption is not fulfilled, a new implementation should be shown to be similarly effective at meeting the functional safety requirement. If the new implementation is shown to be not as effective, the estimation of an increased failure rate and reduced metrics (SFF: Safe Failure Fraction, SPFM: Single-Point Fault Metrics, LFM: Latent Fault Metric) must be re-calculated. The FMEDA can be used to help make this analysis.

MCU safety mechanism defined assumptions on memory, clock, power supply, CPU subsystem and each peripheral.

Some mechanism is achieved by hardware, would not need software operation. But some need software solution to enable these functions, called SR-AD (Safety Related- Application Dependent function). Integrator shall pay attention to this part.

System developer shall check the basic modules (memory, clock, power supply, CPU subsystem) safety mechanism. Some mechanism is achieved by hardware peripherals. For example, while applying system clock setting, system developer shall consider the usage of clock monitor module, this peripheral control is designed in MCAL MCU. System developer should consider enabling CMU in MCU clock setting.

Some mechanisms like memory access protection, these MCU feature would be implemented by OS, system developer should contact BSW OS module supplier for this feature.

System developer shall check project MCU peripheral usage and apply proper mechanism solution to the peripheral usage. Like following examples:

- Integrator shall perform a module related configuration register check after module initialized to Make sure the configured data is targeting value.
- Integrator shall compare the transfer data CRC, check the total receive/transfer data if use DMA module.
- Integrator shall monitor the DMA transfer.
- Integrator shall compare the ADC input to an IC internal known voltage, check the ADC value at start up stage ADC self-function test at start up if ADC module used.

For these MCU safety mechanisms, please refer to “FC7XXX safety manual”. Flagchip provides some software solutions in safety demo. System developer would take them into consideration and develop project safety application or safety complex device driver to fulfil them.

Chapter 9. Known Issues

There are no notable known issues in this version.

Chapter 10. Deviations and Limitations

1. Software specification deviations can be referenced in specific module's User Manual.
2. There are no limitations which are common to all the MCAL modules.

Chapter 11. Revision History

Major changes since the last release.

Version	Date	Description
1.0.0	2024-10-11	Initial Release
1.1.0	2025-01-26	Added description for FC7300F8MDQ
1.2.0	2025-06-03	• Changed Document Structure • Added descriptions for EB tresos installation and activation • Added section Multicore Considerations